



РТУ МИРЭА

Институт информационных технологий

Кафедра цифровой трансформации

Лекция 2. Реляционная структура данных

Фундамент, на котором строятся приложения и который состоит из данных

Relation

Дзгоев Алан Эдуардович

Кандидат технических наук,
доцент кафедры цифровой
трансформации ИИТ
РТУ МИРЭА

Москва,
03.03.2025

Реляционная модель данных



Тема 2_ Реляционная структура данных

1.1. Основные сведения

1.2. Кортежи. Отношения и свойства отношений

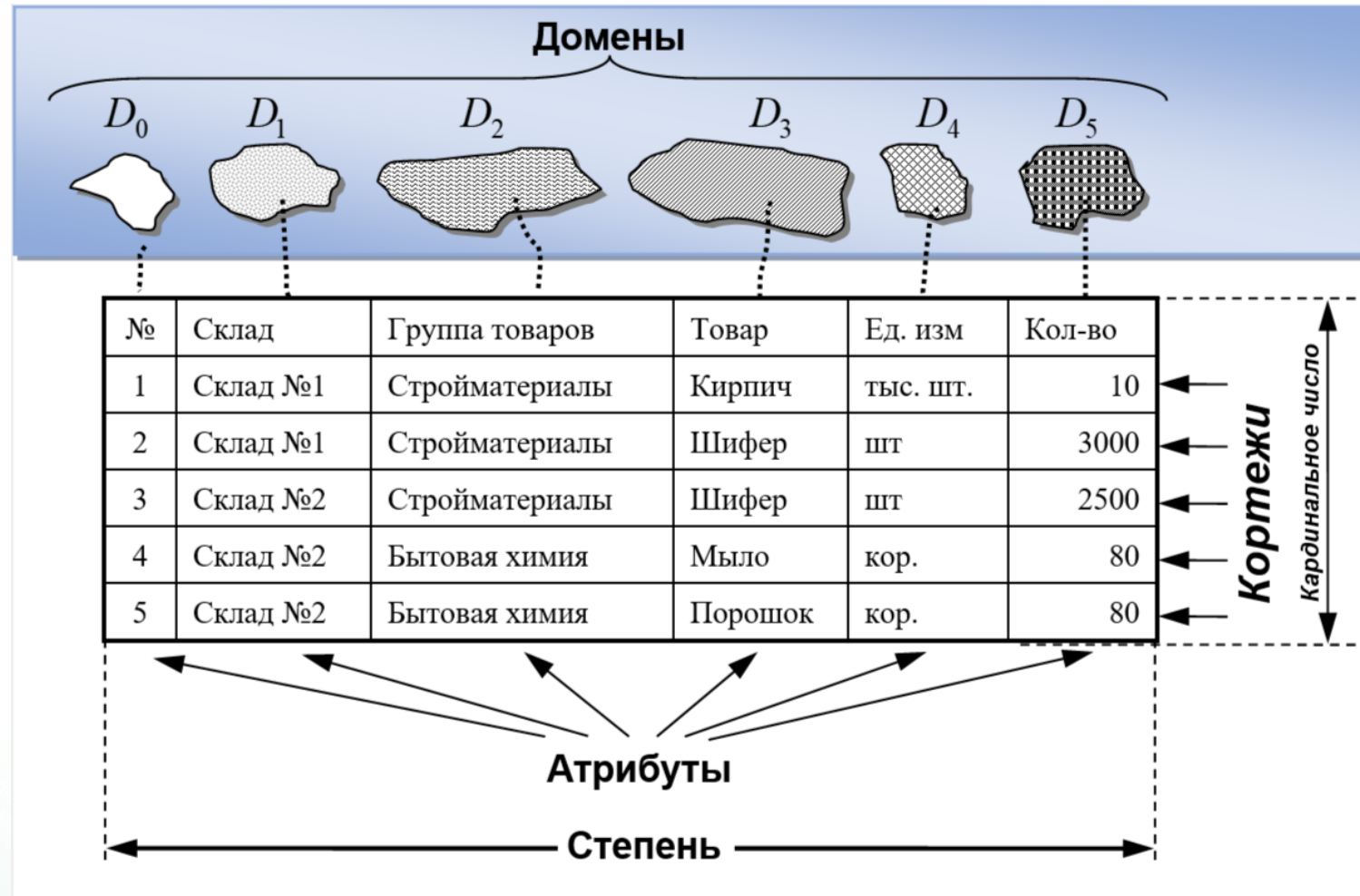
1.3. Ключи.

1.4. Реляционная алгебра. Выборка.

Проекция. Объединение. Пересечение.

Тема 2. Реляционная структура данных

Формальный термин	Неформальный эквивалент
Отношение	Таблица
Кортеж	Строка или запись
Кардинальность	Количество строк
Атрибут	Столбец или поле
Степень отношения	Количество столбцов
Первичный ключ	Уникальный идентификатор
Домен	Множество допустимых значений

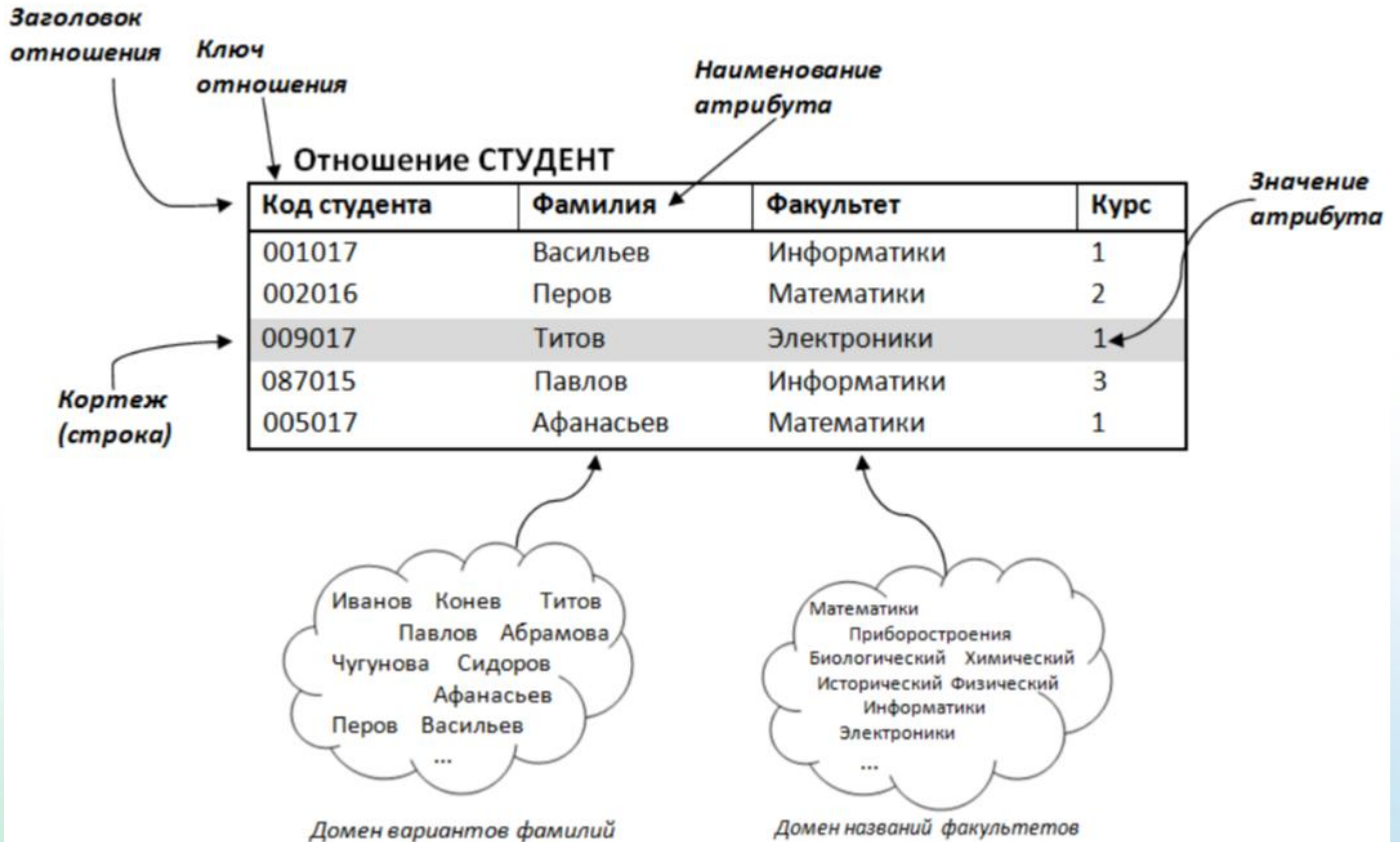


Различная терминология

Отношение	целое	строка		целое		Типы данных
	номер	имя	должность	деньги		Домены
	Табельный номер	Имя	Должность	Оклад	Премия	Атрибуты
	2934	Иванов	инженер	112	40	Кортежи
	2935	Петров	вед. инженер	144	50	
2936	Сидоров	бухгалтер	92	35		

Формальные термины	Альтернатива 1	Альтернатива 2
Relation (отношение)	Table (таблица)	File (файл)
Tuple (кортеж)	Row (строка)	Record (запись)
Attribute (атрибут)	Column (столбец)	Field (поле)

Пример отношения



Пример отношения

Реляционная модель данных

Целое	Строка	Строка	Целое		Типы данных
Номер	Имя	Должность	Деньги		Домены
Табельный номер	Имя	Должность	Оклад	Премия	Атрибуты
2934	Иванов	Инженер	1120	400	Кортежи
2935	Петров	Вед. инженер	1440	500	
2936	Климов	Бухгалтер	920	350	

Ключ



ОТНОШЕНИЕ

Реляционные системы основаны на реляционной модели. Реляционная модель – это абстрактная теория данных, основанная на некоторых областях математики (в основном на теории множеств и логике предикатов).

Реляционная база данных – это такая база данных, в которой данные хранятся в двумерных таблицах, а установленная между ними связь повышает эффективность обработки и уменьшает повторяемость одних и тех же данных.

В общем случае можно считать, что БД включает одну или несколько таблиц, объединенных смысловым содержанием, а также процедурами контроля целостности и обработки информации в интересах решения некоторой прикладной задачи.

Таблица blogdata

id	title	desc	text	img	author	date
1	Чехия	Классно..	Всё нач..	ch.jpg	Е. Попов	18.04.07
2	Москва	Супер в..	Один м...	ms.jpg	Е. Попов	12.02.07

id	name	link	desc
1	Блог Аз	http://azamatus.	Личный блог Азама
2	Блог Ф.	http://photosho..	Блог Фотошоп маст

Таблица friends

id	name	link	desc
1	Блог Аз	http://azamatus.	Личный блог Азама
2	Блог Ф.	http://photosho..	Блог Фотошоп маст



PostgreSQL

text	img	author	date
Всё нач..	/ch.jpg	Е. Попов	18.04.07
Один м...	/ms.jpg	Е. Попов	12.02.07

Смысл отношений (или смысл абстрактных таблиц)

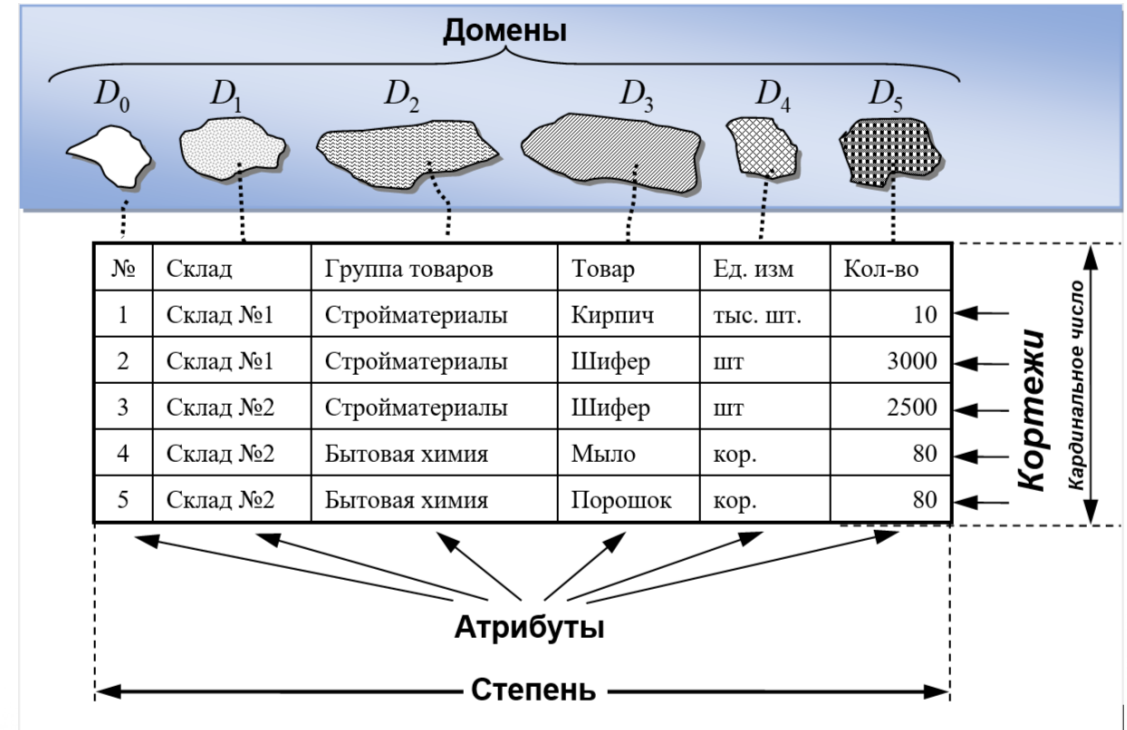
Столбцы в отношениях связаны с **типами данных**.

Реляционная модель включает

«**неограниченный набор типов [данных]**»

Пользователи могут определять собственные типы.

EMP# : EMP#	ENAME : NAME	DEPT# : DEPT#	SALARY : MONEY
E1	Ivanov	D1	40K
E2	Cheng	D1	42K
E3	Yang	D2	30K
E4	Petrov	D2	35K



Служащий с табельным номером E1 и фамилией Иванов работает в отделе с номером D1 и получает зарплату 40 тыс. долл. в год.

Типы – это объекты (множества объектов), которые могут стать предметом обсуждения.

Отношения – это факты (множества фактов), касающиеся объектов, которые могут стать предметом обсуждения.

Пример

Students

Id	FirstName	LastName	GroupId
1	Jack	Jones	2
2	Harry	Miller	1
3	Grace	Evans	3
4	Lily	Wilson	3
5	Joshua	Johnson	1
6	Emily	Taylor	4
7	Charlie	Thomas	4
8	Oliver	Moore	2
9	Jessica	Brown	1

Groups

Id	Name	FacultyId
1	29PR31	1
2	29GR32	2
3	29PPS12	1
4	29GPS21	2



Исторические сведения о реляционной модели

Реляционная модель была впервые предложена Эдгаром Ф. Коддом (E. F. Codd) в статье «A relational model of data for large shared data banks» в 1970 г.

Целью было:

1. Добиться высокой степени независимости приложений от данных (т. е. от изменений внутреннего представления данных: организации файлов, упорядочивания записей и изменения путей доступа).
2. Добиться согласованности данных и решить проблему избыточности данных (было введено понятие нормализованных отношений, о котором речь пойдет в следующих лекциях).

Исследовательские проекты:

- System R в компании IBM, вторая половина 1970-х гг.
 - Разработка языка SQL и первых коммерческих реляционных СУБД
- Проект INGRES в Калифорнийском университете в Беркли (University of California at Berkeley).
INGRES – INteractive Graphics REtrieval System
Майкл Стоунбрейкер (Michael Stonebraker)

A Relational Model of Data for Large Shared Data Banks

E. F. CODD
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information. Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on *n*-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

1.1. INTRODUCTION
This paper is concerned with the application of elementary relation theory to systems which provide shared access to large banks of formatted data. Except for a paper by Childs [1], the principal application of relations to data systems has been to deductive question-answering systems. Levein and Maron [2] provide numerous references to work in this area.
In contrast, the problems treated here are those of *data independence*—the independence of application programs and terminal activities from growth in data types and changes in data representation—and certain kinds of *data inconsistency* which are expected to become troublesome even in nondeductive systems.

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the “connection trap”). Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. DATA DEPENDENCIES IN PRESENT SYSTEMS
The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing dependence, and access path dependence. In some systems these dependencies are not clearly separable from one another.

1.2.1. Ordering Dependence. Elements of data in a data bank may be stored in a variety of ways, some involving no concern for ordering, some permitting each element to participate in one ordering only, others permitting each element to participate in several orderings. Let us consider those existing systems which either require or permit data elements to be stored in at least one total ordering which is closely associated with the hardware-determined ordering of addresses. For example, the records of a file concerning parts might be stored in ascending order by part serial number. Such systems normally permit application programs to assume that the order of presentation of records from such a file is identical to (or is a subordering of) the



Реляционная модель данных для больших совместно используемых банков данных

<https://citforum.ru/database/classics/codd/>

Будущие пользователи больших банков данных должны быть освобождены от необходимости знать организацию данных в машине (внутреннее представление). Нельзя считать удовлетворительной какую-либо службу, если она предоставляет такую информацию. Изменение внутреннего представления данных и даже изменение некоторых аспектов их внешнего представления не должны влиять на работу пользователей за их терминалами и на выполнение большинства прикладных программ. Изменения представления данных часто требуются по причине изменения потока запросов, операций обновления и отчетов, естественного увеличения числа типов хранимой информации.



Christopher Date

Реляционная модель данных предложена сотрудником фирмы IBM Эдгаром Коддом и основывается на понятии отношение (relation).

Отношение представляет собой множество элементов, называемых кортежами.

Реляционная модель данных (РМД) – это способ рассмотрения данных, при котором данные воспринимаются пользователем как взаимосвязанные таблицы и в распоряжении пользователя имеются некоторые операторы, которые генерируют новые таблицы из старых.

Под **таблицами** здесь понимается структура данных, состоящая из строк и столбцов. В этой структуре каждый столбец содержит данные только одного типа, каждая строка состоит из набора значений составляющих ее столбцов.

Под **операторами** понимаются *операции выборки, группировки, соединения и некоторые другие*, результатом которых являются новые таблицы, полученные на основании старых.

Основной **структурой данных в РМД** является отношение (от англ. relation – отношение). Отсюда возникло название модели, основанной на отношениях: такую модель стали называть реляционной моделью данных.

Реляционная модель данных (РМД) некоторой предметной области представляет собой набор отношений, изменяющихся во времени. При создании информационной системы совокупность отношений позволяет хранить данные об объектах предметной области и моделировать связи между ними.

Основные положения реляционной модели

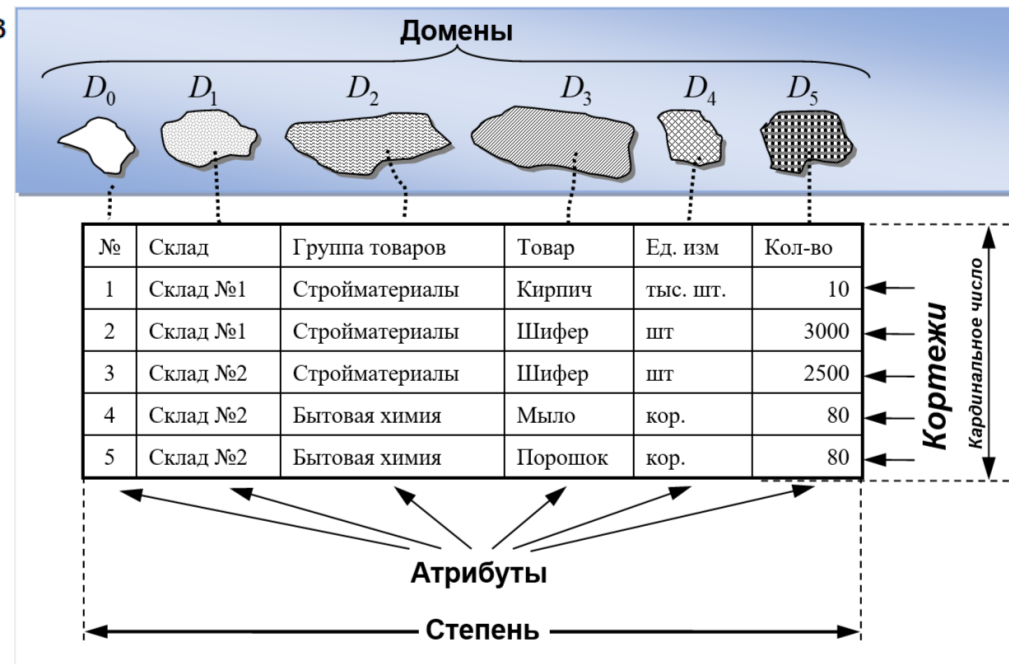
В реляционной системе выполняются как минимум три условия:

- ❑ **Структурный аспект.** Данные в базе *воспринимаются* пользователем, как таблицы (и никак иначе).
- ❑ **Аспект целостности.** Эти таблицы отвечают определенным условиям целостности.
- ❑ **Аспект обработки.** В распоряжении пользователя имеются операторы манипулирования таблицами, которые генерируют новые таблицы на основании уже имеющихся и среди которых есть, по крайней мере, операторы сокращения (restrict), проекции (project) и объединения (join).
- Операция *сокращения* извлекает указанные строки из таблицы (в названии этой операции подразумевается, что число строк ее результата меньше или равно числу строк исходной таблицы). Операцию сокращения иногда называют *выборкой*.
- Операция *проекции* предназначена для извлечения определенных столбцов из таблицы.
- Операция *соединения* предназначена для получения комбинации двух таблиц на основе общих значений в общих столбцах.
- Эти определения являются нестрогими!

СТРОКИ	СТОЛБЦЫ			
		1-поле	2-поле	3-поле
	1-запись			
	2-запись			
	3-запись			
	4-запись			
	5-запись			

Основные сведения

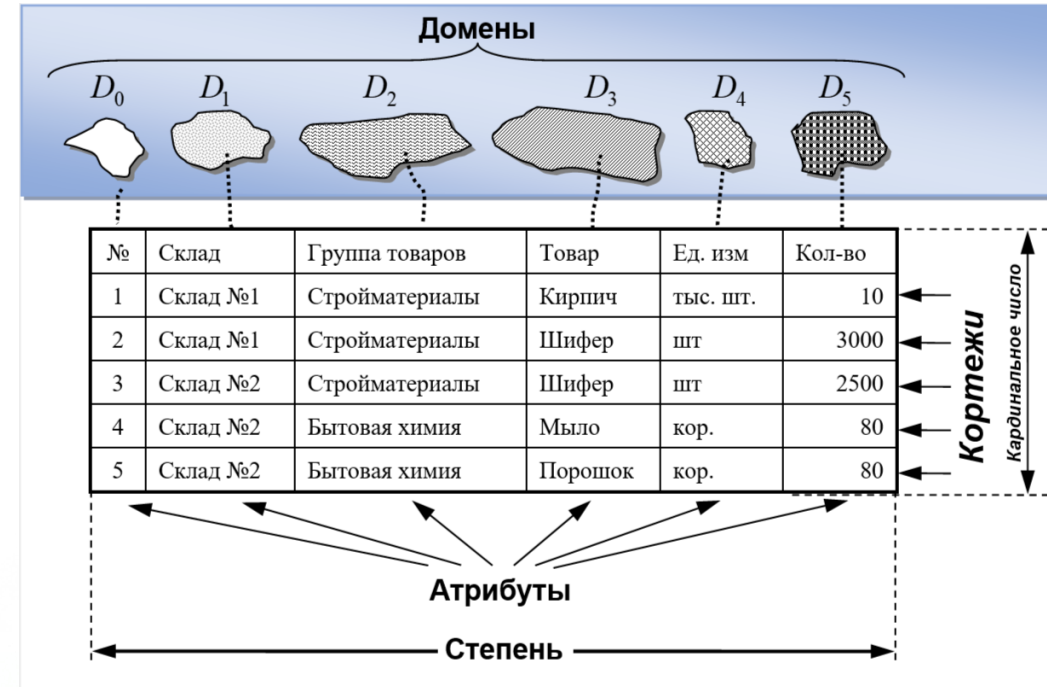
- **Реляционная модель данных** – данные представлены посредством строк в таблицах.
- В теории баз данных эти таблицы называют **отношениями (relations)** – поэтому и базы данных называются **реляционными**. Отношение – это математический термин. При определении свойств таких отношений используется теория множеств. В терминах данной теории строки таблицы будут называться **кортежами (tuples)**, а колонки – **атрибутами**. Отношение имеет заголовок, который состоит из атрибутов, и тело, состоящее из кортежей. Количество атрибутов называется **степенью отношения**, а количество кортежей – **кардинальным числом**.
- Реляционные системы используют *декларативный* подход к получению данных для обработки. Это означает, что требуется только указать, какие данные нужны, но не нужно предписывать способ их получения.
- Реляционная модель требует, чтобы данные воспринимались в виде таблиц. Однако это требование относится только к логической структуре базы данных, т. е. только к внешнему и концептуальному уровням архитектуры ANSI/SPARC. На физическом уровне могут использоваться какие угодно структуры хранения данных и механизмы доступа.



Кортежи

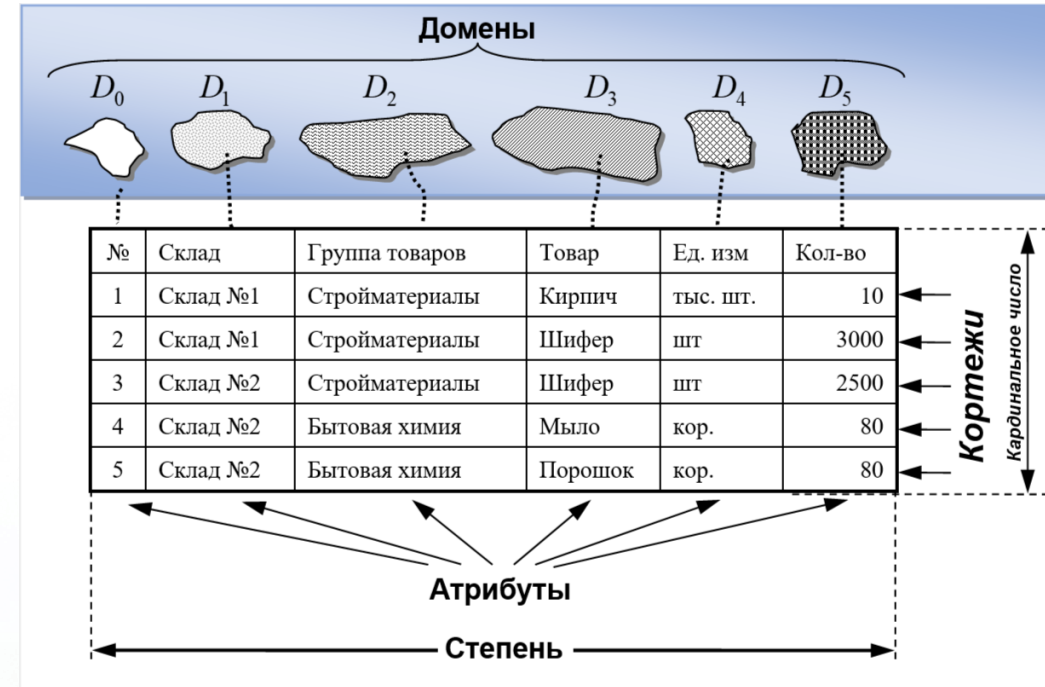
- Если дана коллекция типов T_i ($i = 1, 2, \dots, n$), которые не обязательно все должны быть разными, то значением кортежа (или кратко *кортежем*), определенным с помощью этих типов (назовем его t), является множество упорядоченных троек в форме $\langle A_i, T_i, v_i \rangle$, где A_i — имя атрибута, T_i — имя типа и v_i — значение типа T_i . Кроме того, кортеж t должен соответствовать требованиям:
- Значение n — это **степень**,
- Упорядоченная тройка $\langle A_i, T_i, v_i \rangle$ является **компонентом** t .
- Упорядоченная пара $\langle A_i, T_i \rangle$ представляет собой **атрибут** t и однозначно определяется именем атрибута A_i .
- Значение v_i — это **значение атрибута**, соответствующее атрибуту A_i кортежа t . Тип T_i — это соответствующий **тип атрибута**. Полное множество атрибутов составляет **заголовок кортежа** t .
- Тип кортежа** t определен заголовком t , а сам заголовок и этот тип кортежа имеют такие же атрибуты (и поэтому такие же имена и типы атрибутов) и такую же степень, как t .

MAJOR_P# : P#	MINOR_P# : P#	QTY : QTY
P2	P4	7



Отношения

- Отношение это математический термин. Необходимо различать переменную отношения (relation variable) и значение отношения.
- Каждое отношение, точнее, каждое *значение* отношения, состоит из двух частей: набора пар «*имя_столбца* : *имя_типа*» (**заголовок**) и множества кортежей, согласованных с этим заголовком (**тела**). Кардинальность r отношения определяется как кардинальность этого множества (*кардинальностью множества* называется количество элементов множества).
- Отношение r имеет такие же атрибуты (следовательно, такие же имена и типы атрибутов) и такую же степень, как заголовок.
- Тело отношения r представляет собой множество кортежей, имеющих один и тот же заголовок.



Фундаментальные свойства отношений

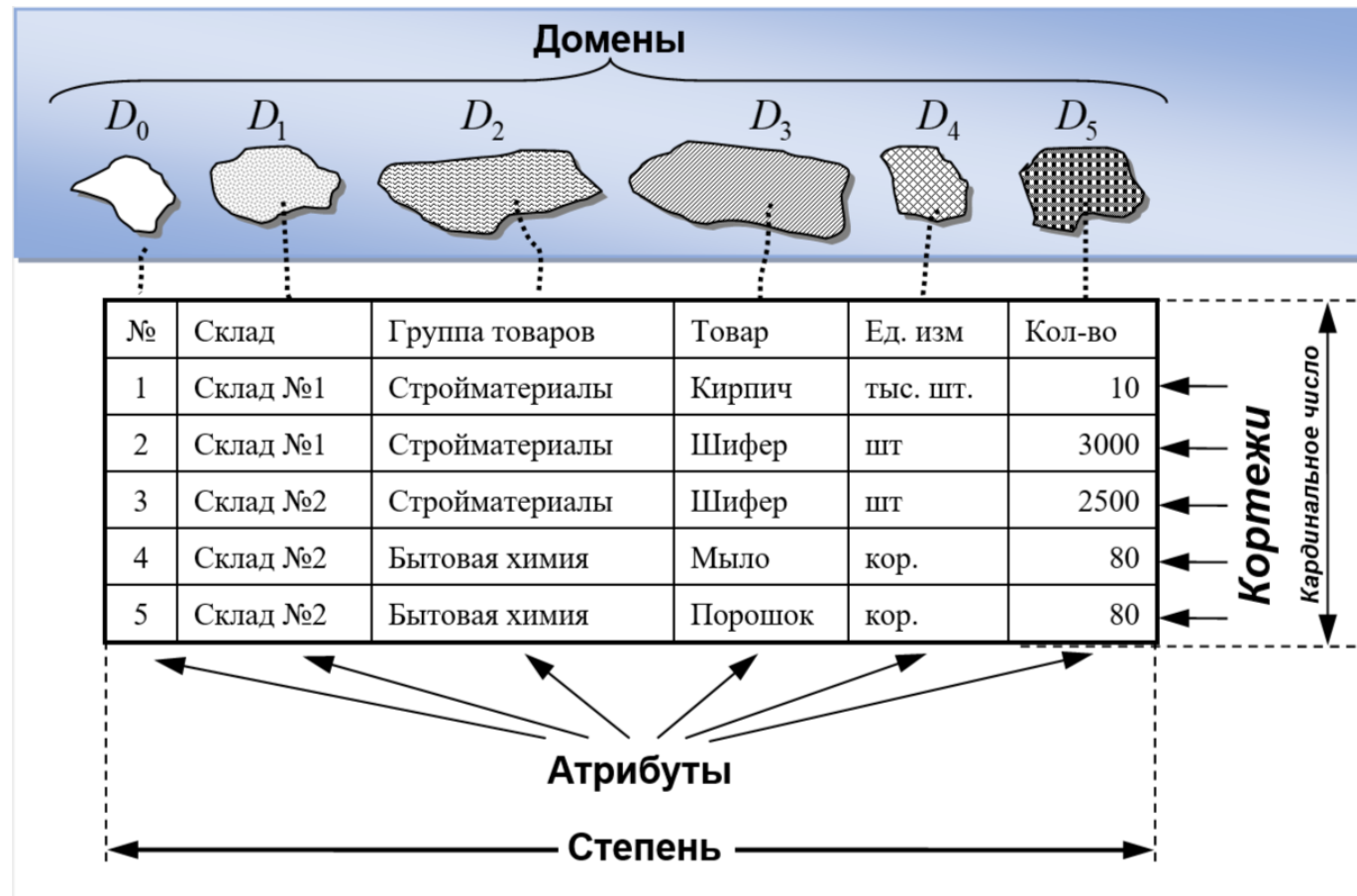
Каждое отношение обладает следующими свойствами:

1. Каждый кортеж содержит только одно значение (соответствующего типа) для каждого атрибута. Атрибуты могут иметь значения в виде отношений.

2. Атрибуты не характеризуются каким-либо упорядочиванием (например, слева направо).

3. Кортежи не характеризуются каким-либо упорядочением (например, сверху вниз).

4. В отношении отсутствуют дубликаты кортежей.

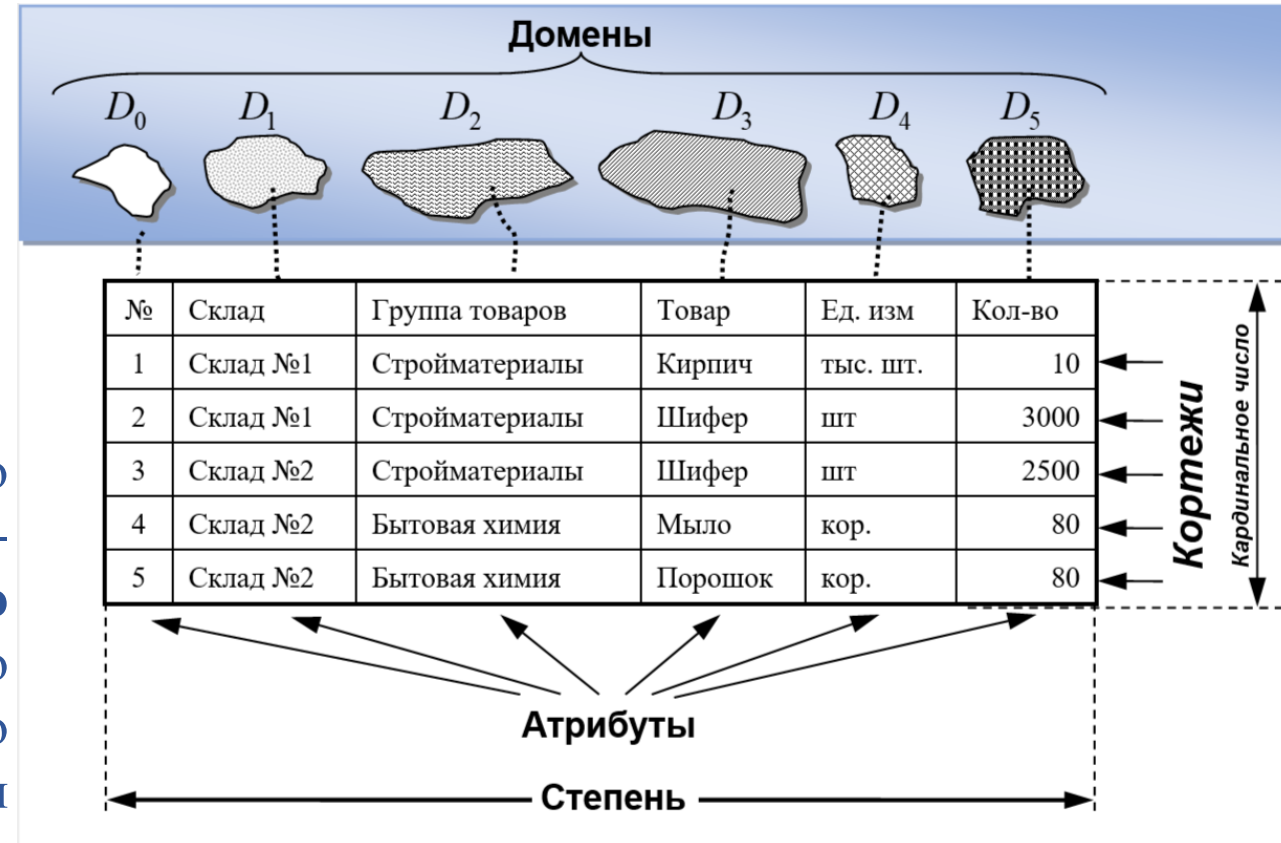


Фундаментальные свойства отношений (подробнее)

1. Отсутствие кортежей-дубликатов

То свойство, что отношения не содержат кортежей-дубликатов, следует из определения отношения как множества кортежей. В классической теории множеств по определению каждое множество состоит из различных элементов.

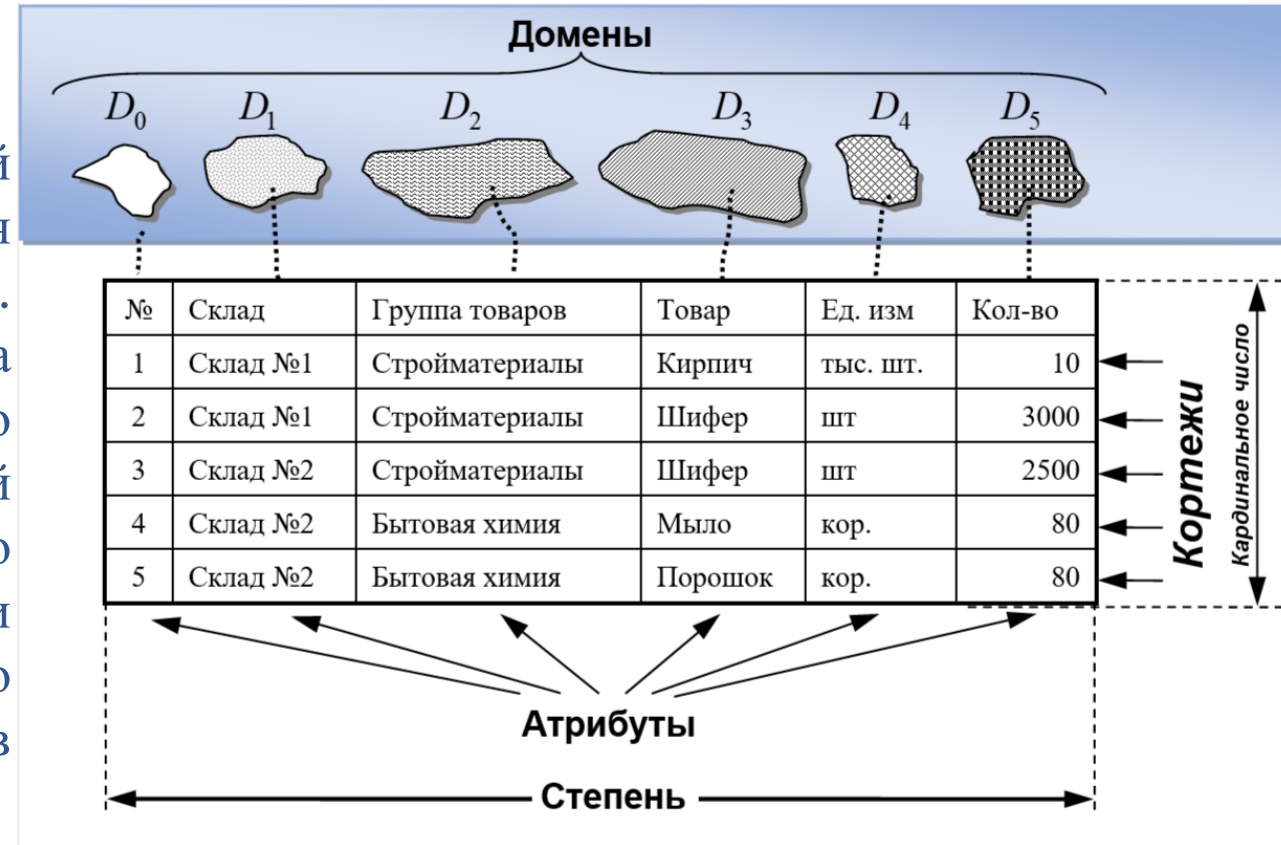
Из этого свойства вытекает наличие у каждого отношения так называемого первичного ключа - набора атрибутов, значения которых однозначно определяют кортеж отношения. Для каждого отношения по крайней мере полный набор его атрибутов обладает этим свойством. Однако при формальном определении первичного ключа требуется обеспечение его "минимальности", т.е. в набор атрибутов первичного ключа не должны входить такие атрибуты, которые можно отбросить без ущерба для основного свойства - однозначно определять кортеж. Понятие *первичного ключа* является исключительно важным в связи с понятием целостности баз данных.



Фундаментальные свойства отношений

2. Отсутствие упорядоченности кортежей

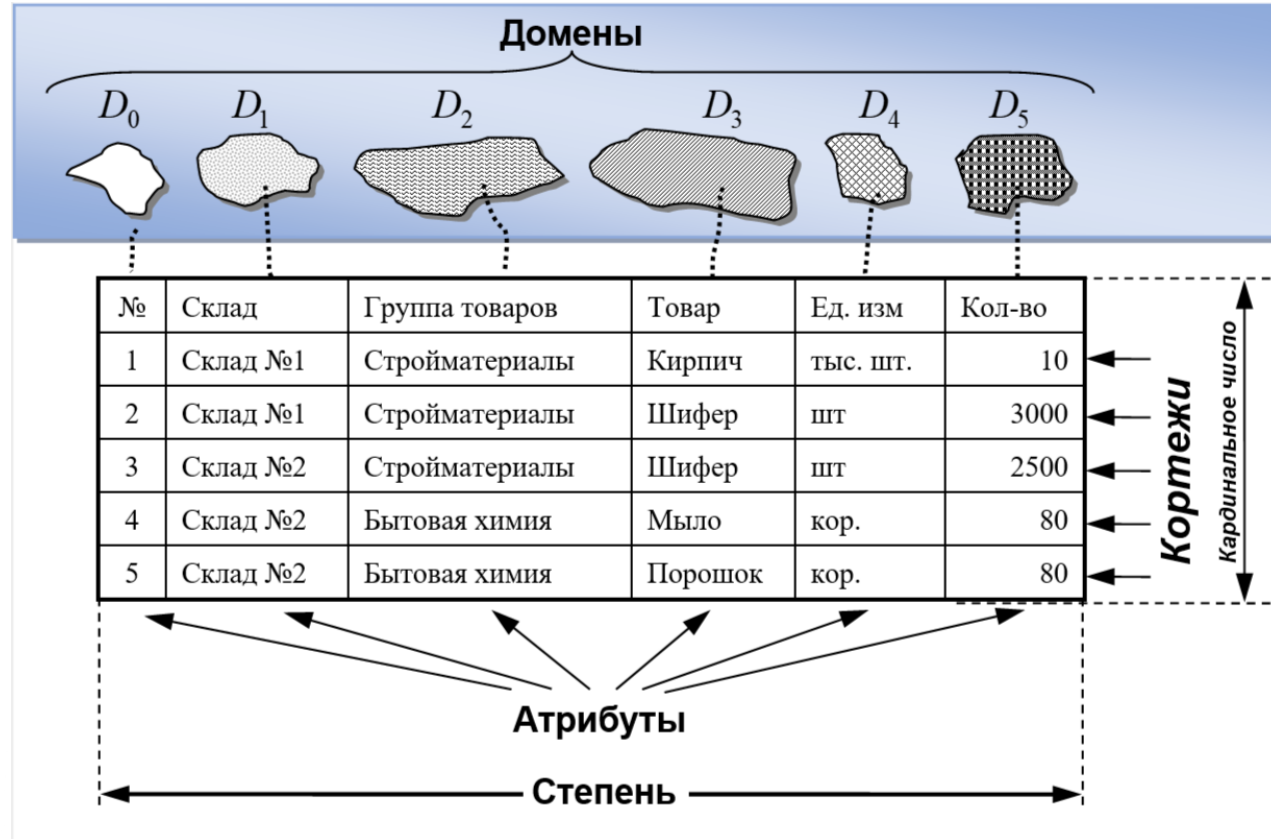
Свойство отсутствия упорядоченности кортежей отношения также является следствием определения отношения-экземпляра как множества кортежей. Отсутствие требования к поддержанию порядка на множестве кортежей отношения дает дополнительную гибкость СУБД при хранении баз данных во внешней памяти и при выполнении запросов к базе данных. Это не противоречит тому, что при формулировании запроса к БД, например, на языке SQL можно потребовать сортировки результирующей таблицы в соответствии со значениями некоторых столбцов.



Фундаментальные свойства отношений

3. Отсутствие упорядоченности атрибутов

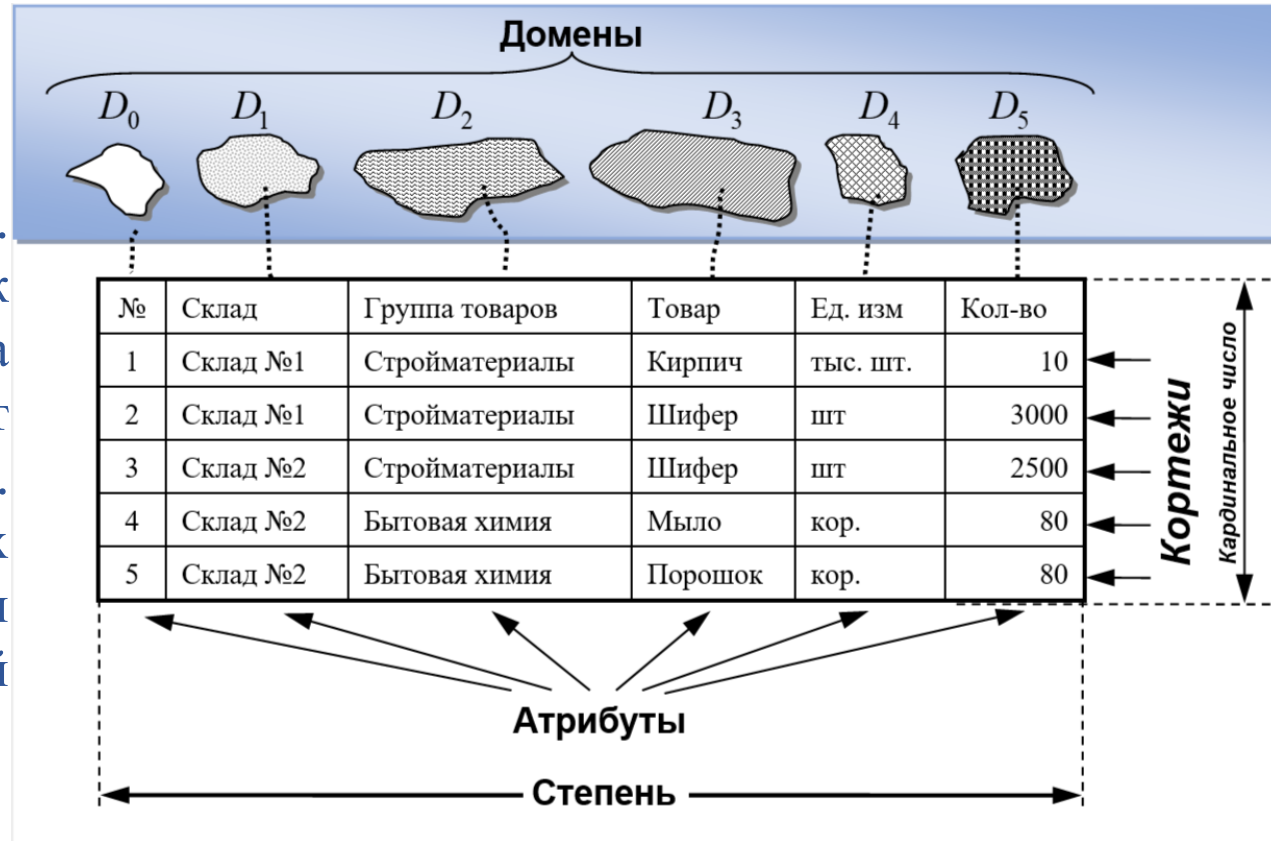
Атрибуты отношений не упорядочены, поскольку по определению схема отношения есть множество пар {имя атрибута, имя домена}. Для ссылки на значение атрибута в кортеже отношения всегда используется имя атрибута. Это свойство теоретически позволяет, например, модифицировать схемы существующих отношений не только путем добавления новых атрибутов, но и путем удаления существующих атрибутов. Однако в большинстве существующих систем такая возможность не допускается, и хотя упорядоченность набора атрибутов отношения явно не требуется, часто в качестве неявного порядка атрибутов используется их порядок в линейной форме определения схемы отношения.



Фундаментальные свойства отношений

4. Атомарность значений атрибутов

Значения всех атрибутов являются атомарными. Это следует из определения домена как потенциального множества значений простого типа данных, т.е. среди значений домена не могут содержаться множества значений (отношения). Принято говорить, что в реляционных базах данных до-пускаются только нормализованные отношения или отношения, представ-ленные в первой нормальной форме.



Примеры ненормализованного(а) и нормализованного(б) отношений

а)

ОТНОШЕНИЕ 1

Код студента	Дисциплина	Оценка
001	Информатика	5
	История	4
	Электротехника	5
	Физика	4
002	Информатика	3
	Базы данных	4
	Физика	4
003	Математика	5
	Физика	5
	Электротехника	3

б)

ОТНОШЕНИЕ 2

Код студента	Дисциплина	Оценка
001	Информатика	5
001	История	4
001	Электротехника	5
001	Физика	4
002	Информатика	3
002	Базы данных	4
002	Физика	4
003	Математика	5
003	Физика	5
003	Электротехника	3

Основные понятия реляционной модели

Ограничения

При работе с базами данных часто приходится следовать различным **ограничениям**, которые могут быть обусловлены спецификой конкретной предметной области:

- номер зачетной книжки состоит из пяти цифр и не может быть отрицательным
- серия документа, удостоверяющего личность, является четырехзначным числом, а номер документа, удостоверяющего личность — шестизначным числом
- номер семестра может принимать только два значения — 1 (осенний семестр) и 2 (весенний семестр)
- оценка может принимать только три значения — 3 (удовлетворительно), 4 (хорошо) и 5 (отлично)

БАЗА ДАННЫХ ОНЛАЙН-ШКОЛЫ

products

id	name	type_id	price
1	Основы искусственного интеллекта	1	15000
2	Технологии обработки больших данных	1	50000
3	Программирование глубоких нейронных сетей	1	30000
4	Нейронные сети для анализа текстов	1	50000
5	Нейронные сети для анализа изображений	1	50000
6	Инженерия искусственного интеллекта	1	60000
7	Как стать DataScientist'ом	2	0
8	Планирование карьеры в DataScience	2	2000
9	Области применения нейросетей: в какой развивать экспертизу	2	4000
10	Программирование глубоких нейронных сетей на Python	3	1000
11	Математика для DataScience	3	2000
12	Основы визуализации данных	3	500

product_types

id	type_name
1	Онлайн-курс
2	Вебинар
3	Книга
4	Консультация

БАЗА ДАННЫХ ОНЛАЙН-ШКОЛЫ

products

id	name	type_id	price
1	Основы искусственного интеллекта	1	15000
2	Технологии обработки больших данных	1	50000
3	Программирование глубоких нейронных сетей	1	30000
4	Нейронные сети для анализа текстов	1	50000
5	Нейронные сети для анализа изображений	1	50000
6	Инженерия искусственного интеллекта	1	60000
7	Как стать DataScientist'ом	2	0
8	Планирование карьеры в DataScience	2	2000
9	Области применения нейросетей: в какой развивать экспертизу	2	4000
10	Программирование глубоких нейронных сетей на Python	3	1000
11	Математика для DataScience	3	2000
12	Основы визуализации данных	3	500

product_types

id	type_name
1	Онлайн-курс
2	Вебинар
3	Книга
4	Консультация

БАЗА ДАННЫХ ОНЛАЙН-ШКОЛЫ

products

id	name	type_id	price
1	Основы искусственного интеллекта	1	15000
2	Технологии обработки больших данных	1	50000
3	Программирование глубоких нейронных сетей	1	30000
4	Нейронные сети для анализа текстов	1	50000
5	Нейронные сети для анализа изображений	1	50000
6	Инженерия искусственного интеллекта	1	60000
7	Как стать DataScientist'ом	2	0
8	Планирование карьеры в DataScience	2	2000
9	Области применения нейросетей: в какой развивать экспертизу	2	4000
10	Программирование глубоких нейронных сетей на Python	3	1000
11	Математика для DataScience	3	2000
12	Основы визуализации данных	3	500

product_types

id	type_name
1	Онлайн-курс
2	Вебинар
3	Книга
4	Консультация

БАЗА ДАННЫХ ОНЛАЙН-ШКОЛЫ

products

id	name	type_id	price
1	Основы искусственного интеллекта	1	15000
2	Технологии обработки больших данных	1	50000
3	Программирование глубоких нейронных сетей	1	30000
4	Нейронные сети для анализа текстов	1	50000
5	Нейронные сети для анализа изображений	1	50000
6	Инженерия искусственного интеллекта	1	60000
7	Как стать DataScientist'ом	2	0
8	Планирование карьеры в DataScience	2	2000
9	Области применения нейросетей: в какой развивать экспертизу	2	4000
10	Программирование глубоких нейронных сетей на Python	3	1000
11	Математика для DataScience	3	2000
12	Основы визуализации данных	3	500

product_types

id	type_name
1	Онлайн-курс
2	Вебинар
3	Книга
4	Консультация

Таблица «Студенты» (students)

Номер зачетной книжки (record_book)	Ф. И. О. (name)	Серия паспорта (psp_ser)	Номер паспорта (psp_num)
55500	Иванов Иван Петрович	0402	645327
55800	Климов Андрей Иванович	0402	673211
55865	Новиков Николай Юрьевич	0202	554390

Таблица «Успеваемость» (progress)

Номер зачетной книжки (record_book)	Предмет (discipline)	Учебный год (acad_year)	Семестр (term)	Оценка (mark)
55500	Физика	2017/2018	1	5
55500	Математика	2017/2018	1	4
55800	Физика	2017/2018	1	4
55800	Физика	2017/2018	2	5

Ключи

- Ключи служат для идентификации строк в таблицах и для связи таблиц между собой.
- Потенциальный ключ — это комбинация атрибутов таблицы, позволяющая **уникальным** образом идентифицировать строки в ней.
- Ключ может состоять и только лишь из одного атрибута таблицы.
Например, в таблице «Студенты» таким идентификатором может быть атрибут «Номер зачетной книжки».
В качестве потенциального ключа данной таблицы могут также служить два ее атрибута, взятые вместе: «Серия паспорта» и «Номер паспорта». Ни один из них в отдельности не может использоваться в качестве уникального идентификатора. В таком случае ключ будет **составным**.
- Потенциальный ключ должен быть **не избыточным**, т. е. никакое подмножество атрибутов, входящих в него, не должно обладать свойством уникальности.

Ключи (продолжение)

- При наличии в таблице более одного потенциального ключа один из них выбирается в качестве так называемого **первичного** ключа, а остальные будут являться **альтернативными** ключами.
- Предположим, что в таблице «Студенты» нет строки с номером зачетной книжки 55900, тогда включать строку с таким номером зачетной книжки в таблицу «Успеваемость» не имеет смысла. Таким образом, значения столбца «Номер зачетной книжки» в таблице «Успеваемость» должны быть согласованы со значениями такого же столбца в таблице «Студенты».
- Атрибут «Номер зачетной книжки» в таблице «Успеваемость» является примером того, что называется **внешним ключом**. Таблица, содержащая внешний ключ, называется **ссылающейся** таблицей (referencing table). Таблица, содержащая соответствующий потенциальный ключ, называется **ссылочной** (целевой) таблицей (referenced table). В таких случаях говорят, что внешний ключ ссылается на потенциальный ключ в ссылочной таблице.
- Внешний ключ может быть составным, т. е. может включать более одного атрибута. Внешний ключ не обязан быть уникальным.

Основные понятия реляционной модели

Первичный ключ. Чтобы отличать одну строку от другой используется понятие первичного ключа. Первичным ключом называется столбец или группа столбцов таблицы, значения которых уникальным образом идентифицируют каждую строку таблицы.
У таблицы может быть только один первичный ключ.

Таблица «Студенты» (students)

Номер зачетной книжки (record_book)	Ф. И. О. (name)	Серия паспорта (psp_ser)	Номер паспорта (psp_num)
55500	Иванов Иван Петрович	0402	645327
55800	Климов Андрей Иванович	0402	673211
55865	Новиков Николай Юрьевич	0202	554390

Таблица «Успеваемость» (progress)

Номер зачетной книжки (record_book)	Предмет (discipline)	Учебный год (acad_year)	Семестр (term)	Оценка (mark)
55500	Физика	2017/2018	1	5
55500	Математика	2017/2018	1	4
55800	Физика	2017/2018	1	4
55800	Физика	2017/2018	2	5

Таблица «Студенты» (students)

Номер зачетной книжки (record_book)	Ф. И. О. (name)	Серия паспорта (psp_ser)	Номер паспорта (psp_num)
55500	Иванов Иван Петрович	0402	645327
55800	Климов Андрей Иванович	0402	673211
55865	Новиков Николай Юрьевич	0202	554390

Таблица «Успеваемость» (progress)

Номер зачетной книжки (record_book)	Предмет (discipline)	Учебный год (acad_year)	Семестр (term)	Оценка (mark)
55500	Физика	2017/2018	1	5
55500	Математика	2017/2018	1	4
55800	Физика	2017/2018	1	4
55800	Физика	2017/2018	2	5

Ключи обычно используют для достижения следующих целей:

1. исключения дублирования значений в ключевых атрибутах (остальные атрибуты в расчет не принимаются);
2. упорядочения кортежей. Возможно упорядочение по возрастанию или убыванию значений всех ключевых атрибутов, а также смешанное упорядочение (по одним – возрастание, а по другим – убывание);
3. ускорения работы с кортежами отношения;
4. организации связанных таблиц.



Реляционная модель накладывает на внешние ключи ограничение для обеспечения целостности данных, называемое **ссылочной целостностью**. Это означает, что каждому значению внешнего ключа должны соответствовать строки в связываемых отношениях.

Основные понятия реляционной модели (4)

Ссылочная целостность

- Проблема обеспечения того, чтобы база данных не содержала неверных значений внешних ключей, известна как **проблема ссылочной целостности**. Ограничение, согласно которому значения внешних ключей должны соответствовать значениям потенциальных ключей, называется **ограничением ссылочной целостности** (ссылочным ограничением).
- Обеспечением выполнения ограничений ссылочной целостности занимается СУБД, а от разработчика требуется лишь указать атрибуты, служащие в качестве внешних ключей.
- При проектировании баз данных часто предусматривается, что при удалении строки из ссылочной таблицы соответствующие строки из ссылающейся таблицы должны быть также удалены, а при изменении значения столбца, на который ссылается внешний ключ, должны быть изменены значения внешнего ключа в ссылающейся таблице. Этот подход называется **каскадным удалением (обновлением)**.

Таблица «Студенты» (students)

Номер зачетной книжки (record_book)	Ф. И. О. (name)	Серия паспорта (psp_ser)	Номер паспорта (psp_num)
55500	Иванов Иван Петрович	0402	645327
55800	Климов Андрей Иванович	0402	673211
55865	Новиков Николай Юрьевич	0202	554390

Таблица «Успеваемость» (progress)

Номер зачетной книжки (record_book)	Предмет (discipline)	Учебный год (acad_year)	Семестр (term)	Оценка (mark)
55500	Физика	2017/2018	1	5
55500	Математика	2017/2018	1	4
55800	Физика	2017/2018	1	4
55800	Физика	2017/2018	2	5

Сокращение Таблица «Студенты» (students)

Номер зачетной книжки	Ф. И. О.	Серия паспорта	Номер паспорта
55500	Иванов Иван Петрович	0402	645327

Проекция Таблица «Успеваемость» (progress)

Предмет	Учебный год	Семестр	Оценка
Физика	2017/2018	1	5
Математика	2017/2018	1	4
Физика	2017/2018	1	4
Физика	2017/2018	2	5

Соединение Таблицы «Студенты» (students) и «Успеваемость» (progress)

Номер зачетной книжки	Ф. И. О.	Серия паспорта	Номер паспорта	Предмет	Учебный год	Семестр	Оценка
55500	Иванов Иван Петрович	0402	645327	Физика	2017/2018	1	5
55500	Иванов Иван Петрович	0402	645327	Математика	2017/2018	1	4
55800	Климов Андрей Иванович	0402	673211	Физика	2017/2018	1	4
55800	Климов Андрей Иванович	0402	673211	Физика	2017/2018	2	5

Замкнутость реляционной системы

- Свойство **замкнутости** реляционных систем означает, что результат выполнения операции имеет тот же тип, что и объекты, над которыми проводилась операция (все они являются отношениями). Но это значит, что к *результатам операций можно снова применить какую-либо операцию*.
- Например, можно сформировать проекцию соединения, соединение двух сокращений, сокращение проекции и т.д. Другими словами, можно использовать **вложенные реляционные выражения**, т.е. выражения, в которых операнды представлены выражениями, а не простыми именами таблиц.
- Значения переменных отношения изменяются с помощью операций **реляционного присваивания**, причем привычные нам операции обновления **INSERT, UPDATE** и **DELETE** можно считать сокращенной формой записи операций реляционного присваивания определенных типов.

Базовые отношения и представления

- Исходные (заданные) переменные отношения называются **базовыми переменными отношения**, а присвоенные им значения называются **базовыми** отношениями. Отношение, которое получено или может быть получено из базового отношения в результате выполнения каких-либо реляционных выражений, называется **производным** отношением.
- Реляционные системы обычно поддерживают еще один вид именованных переменных отношения, называемых **представлениями**. В любой конкретный момент их значение является *производным* отношением.
- Выражение, фактически определяющее представление, не вычисляется, а просто *запоминается* системой.
- В реляционной модели представления являются одним из способов обеспечения *логической независимости от данных*.
- Базовые переменные отношения «реально существуют» в том смысле, что они воплощают в себе данные, которые действительно хранятся в базе данных.
- Представления, наоборот, «реально не существуют», а просто предоставляют различные способы просмотра «реальных» данных.

Реляционная алгебра

Алгебра — множество объектов с заданной на нем совокупностью операций, замкнутых относительно этого множества, называемого **основным множеством**.

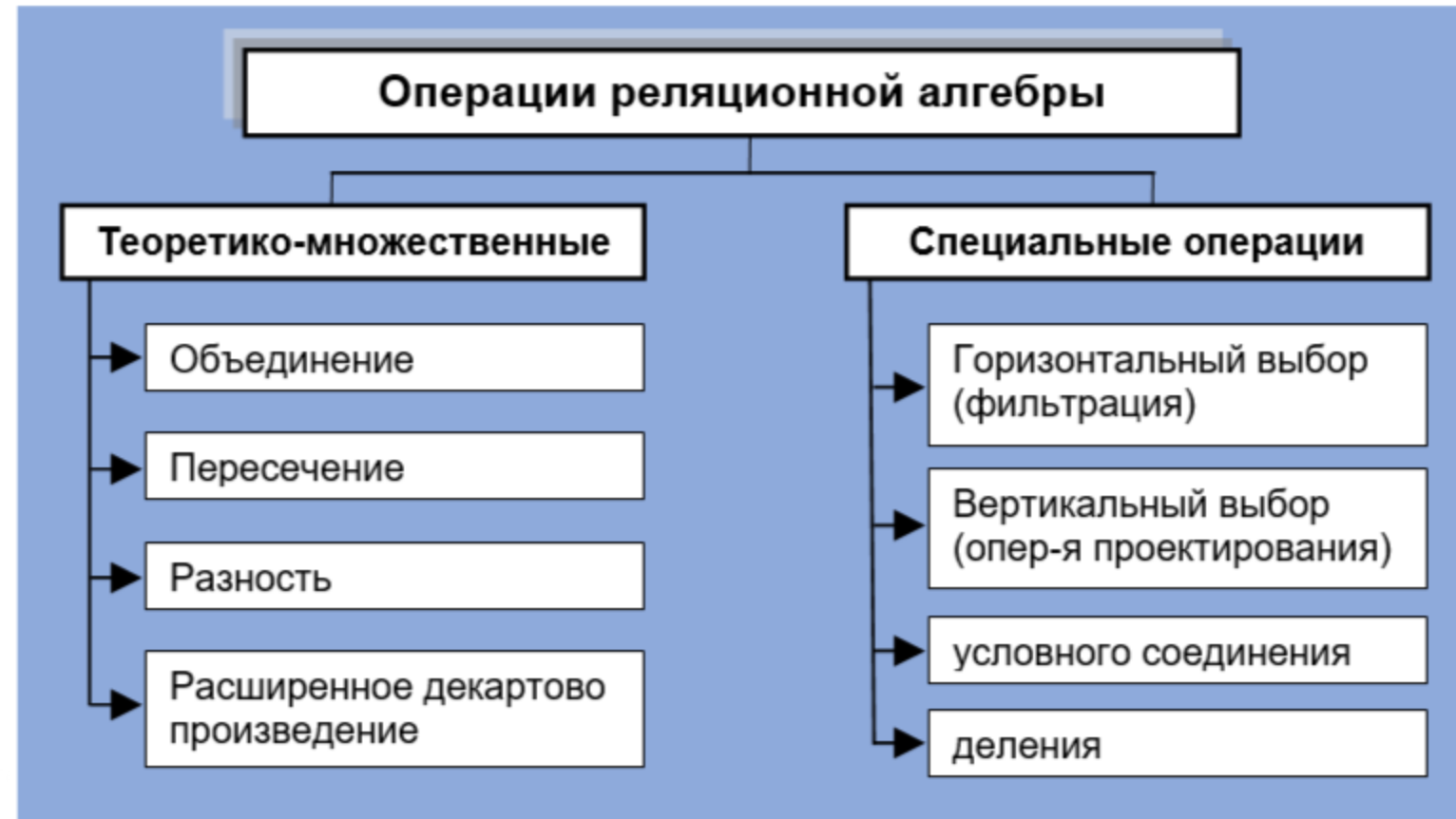
Реляционная алгебра — множество отношений с замкнутым на нем множеством операций над отношениями.

Основным множеством в реляционной алгебре является множество отношений.

Э.Ф. Кодд предложил 8 операций над отношениями.

В общем случае это множество операций является избыточным, т.к. одни операции можно выразить через другие.

Однако, именно такой минимальный состав операций был выбран для того, чтобы обеспечить максимальное удобство при работе с отношениями.



Реляционная алгебра и реляционное исчисление

- Важной составной частью реляционной модели является манипуляционный механизм, т. е. языки запросов, позволяющие извлекать и обновлять данные.
- Это реляционная алгебра и реляционное исчисление предложенные Э. Коддом в 1971 г.
- Неформально можно определить реляционную алгебру как *высокоуровневый процедурный* язык: она может использоваться для того, чтобы указать СУБД, как сформировать новое отношение из одного или более отношений в базе данных.
- Аналогично реляционное исчисление можно описать как *непроцедурный* язык: оно может использоваться для того, чтобы сформулировать определение нового отношение в терминах одного или более отношений, существующих в базе данных.
- Реляционная алгебра и реляционное исчисление формально эквивалентны друг другу, т. е. для каждого выражения реляционной алгебры существует эквивалентное выражение реляционного исчисления (и наоборот).

- Реляционная алгебра и реляционное исчисление являются формальными языками, они не являются дружественными к пользователю.
- Они имеют значение поскольку:
 - Иллюстрируют базовые операции, требующиеся от любого языка манипулирования данными (DML)
 - Выступают в роли стандарта для сравнения других реляционных языков
- Реляционное исчисление используется для оценки мощности реляционных языков. Язык, который позволяет получить любое отношение, которое можно получить с помощью реляционного исчисления, называется **реляционно-полным**.
- Реальные языки зачастую не только являются реляционно-полными, но и имеют большую выразительность, чем реляционная алгебра и реляционное исчисление, за счет наличия дополнительных операций, таких как вычисления, суммирование, упорядочивание.

Реляционная алгебра

- Реляционная алгебра — это коллекция *операций*, которые принимают отношения в качестве операндов и возвращают отношение в качестве результата.
- Результат выполнения одной операции может использоваться в качестве входных данных (операнда) другой операции. Поэтому возможно использование вложенных операций. Это свойство называется **замкнутостью (closure)**.
- Алгебра, определенная Э. Коддом, включала две группы по 4 операции:
- Традиционные операции с множествами — объединение (Union), пересечение (Intersection), разность (Set difference) и декартово произведение (Cartesian product)
- Специальные реляционные операции — сокращение (Restriction), или выборка (Selection), проекция (Projection), соединение (Join) и деление (Division)
- Эти операции можно разделить на базовые (выборка, проекция, декартово произведение, объединение и разность) и дополнительные (соединение, пересечение и деление).
- Операции выборки и проекции являются унарными (работают с одним отношением), остальные операции — бинарные (работают с двумя отношениями).
- Отметим, что все эти операции предназначены *только для чтения* (т. е. они «читают», но не обновляют свои операнды).

Выборка (Selection), или сокращение (Restriction)

- Эта операция выполняется на одном отношении R и определяет отношение, которое содержит только те кортежи исходного отношения R, которые удовлетворяют заданному условию (предикату).
- К. Дейт использует такое обозначение: $R \text{ WHERE } p$
- Некоторые другие авторы — такое: $\sigma_{\text{predicate}}(R)$
- Выражение p — это **предикат**, или **условие сокращения**



Примеры.

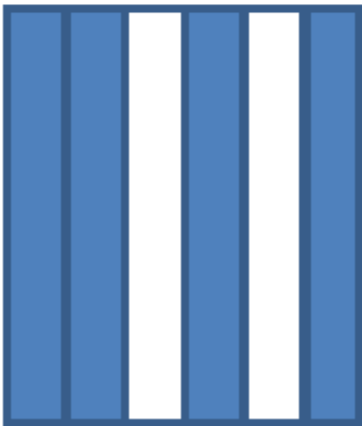
Students WHERE record_book = 55500

Students WHERE record_book = 55500 OR record_book = 55800

Progress WHERE acad_year = '2017/2018' AND term = 1

Проекция (Projection)

- Эта операция выполняется на одном отношении R , которое имеет атрибуты $X, Y, \dots, Z$ (и, возможно, другие атрибуты) и определяет отношение, которое содержит вертикальное подмножество исходного отношения R , извлекая значения указанных атрибутов и отбрасывая дубликаты полученных кортежей. В таком случае **проекция** отношения R по атрибутам $X, Y, \dots, Z$ определяется с помощью следующего выражения (К. Дейт): $\mathbf{R \{ X, Y, \dots, Z \}}$.
- Его заголовок формируется из заголовка отношения R путем удаления всех атрибутов, не указанных в множестве $\{ X, Y, \dots, Z \}$.
- Некоторые авторы используют обозначение: $\mathbf{\Pi_{a_1 \dots, a_n}(R)}$



Пример.

Progress { acad_year, term }

Учебный год (acad_year)	Семестр (term)
2017/2018	1
2017/2018	2

Проекция (Projection)

Это унарная операция (выполняемая над одним отношением), служащая для выбора подмножества атрибутов (столбцов) из отношения R. Она уменьшает арность результирующего отношения (количество столбцов) и может уменьшить мощность результирующего отношения (количество строк) за счёт исключения одинаковых кортежей.

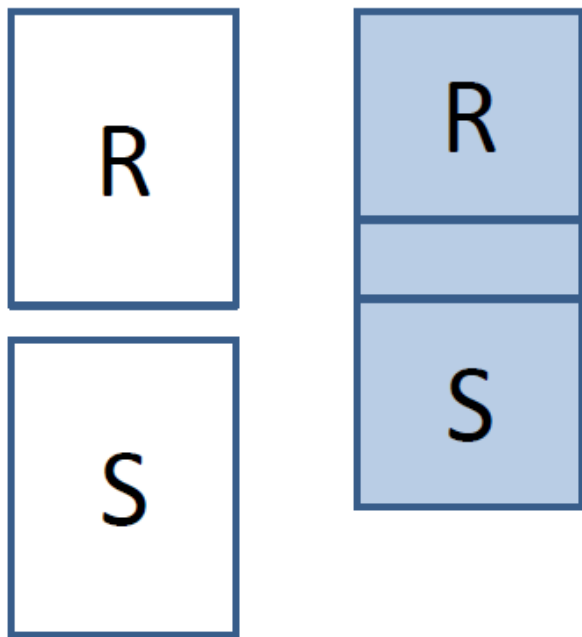
Пример. Для отношение R(A,B,C), проекция $\pi_{A,C}(R)$
(по атрибутам A и C) будет как на рис.:

<u>Отношение R</u>		
A	B	C
a	b	c
c	a	d
c	b	d

<u>Проекция $\pi_{A,C}(R)$</u>	
A	C
a	c
c	d

Объединение (Union)

- Объединением отношений R и S является отношение того же типа с телом, которое состоит из всех кортежей, присутствующих в R или в S , или в обоих отношениях. При этом кортежи-дубликаты устраняются.
- Отношения R и S должны принадлежать к **одному типу**. Они должны быть совместимыми по объединению (union compatible), т. е. иметь одинаковые заголовки.
- К. Дейт использует такое обозначение: $R \text{ UNION } S$
- Некоторые другие авторы — такое: $R \cup S$



Пример.

$\text{Students1} \text{ UNION } \text{Students2}$

(предположим, что оба эти отношения имеют тот же тип, что и отношение Students)

Объединение (Union) пример

Объединением двух отношений называется отношение, содержащее множество кортежей, принадлежащих либо первому, либо второму исходным отношениям, либо обоим отношениями одновременно.

Пусть заданы два отношения

$$R_1 = \{ r_1 \}, R_2 = \{ r_2 \},$$

где r_1 и r_2 - соответственно кортежи отношений R_1 и R_2 , то объединение

$$R_1 \cup R_2 = \{ r \mid r \in R_1 \vee r \in R_2 \}.$$

где r — кортеж нового отношения; $\cup$ - операция логического «ИЛИ».

Рассмотрим пример данной операции.

Предположим, имеются отношения R_1 и R_2 , которые содержат соответственно перечни деталей, изготавливаемых соответственно на первом и втором участках цеха. Отношение R_3 содержит общий перечень деталей, изготавливаемых в цеху.

R_1

Шифр	Название детали
11073	Гайка М1
11075	Гайка М2
11076	Гайка М3
11033	Болт М1
11006	Болт М3
13063	Шайба М1
13066	Шайба М3

R_2

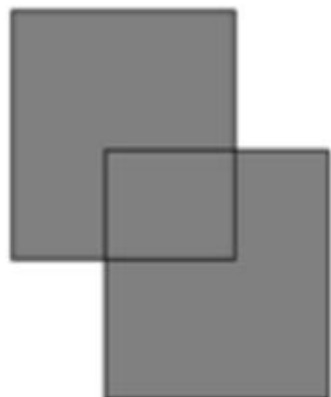
Шифр	Название детали
11073	Гайка М1
11076	Гайка М3
11077	Гайка М4
11004	Болт М2
11006	Болт М3

R_3

Шифр	Название детали
11073	Гайка М1
11075	Гайка М2
11076	Гайка М3
11033	Болт М1
11006	Болт М3
13063	Шайба М1
13066	Шайба М3
11077	Гайка М4
11004	Болт М2

$$R_3 = R_1 \cup R_2$$

Объединение (Union) пример



$R1 \cup R2$

R1

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C3	Титов	Тольятти
C4	Павлов	Москва

R2

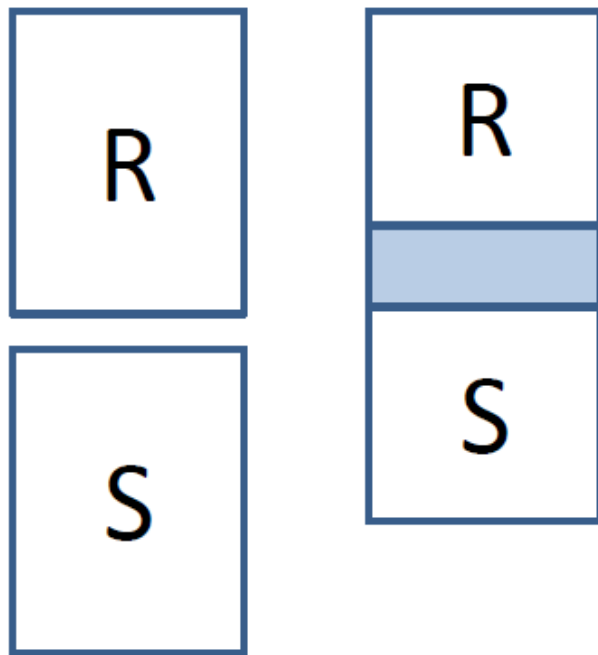
Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C5	Сидоров	Тольятти
C6	Афанасьев	Москва

R (R1 UNION R2)

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C3	Титов	Тольятти
C4	Павлов	Москва
C5	Сидоров	Тольятти
C6	Афанасьев	Москва

Пересечение (Intersection)

- Пересечением отношений R и S является отношение того же типа с телом, которое состоит из всех кортежей, присутствующих одновременно в R и в S .
- Отношения R и S должны принадлежать к **одному типу**. Они должны быть совместимыми по объединению (union compatible), т. е. иметь одинаковые заголовки.
- К. Дейт использует такое обозначение: $R \text{ INTERSECT } S$
- Некоторые другие авторы — такое: $R \cap S$



Пример.

Students1 INTERSECT Students2

(предположим, что оба эти отношения имеют тот же тип, что и отношение Students)

Пересечение (Intersection)

R₁

Шифр	Название детали
11073	Гайка М1
11075	Гайка М2
11076	Гайка М3
11033	Болт М1
11006	Болт М3
13063	Шайба М1
13066	Шайба М3

R₂

Шифр	Название детали
11073	Гайка М1
11076	Гайка М3
11077	Гайка М4
11004	Болт М2
11006	Болт М3

R₄

Шифр	Название детали
11073	Гайка М1
11076	Гайка М3
11006	Болт М3

$$R_4 = R_1 \cap R_2$$

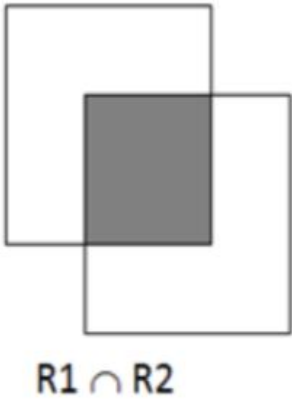
Пересечением отношений называется отношение, которое содержит множество кортежей, принадлежащих одновременно и первому и второму отношениям.

R1

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C3	Титов	Тольятти
C4	Павлов	Москва

R2

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C5	Сидоров	Тольятти
C6	Афанасьев	Москва

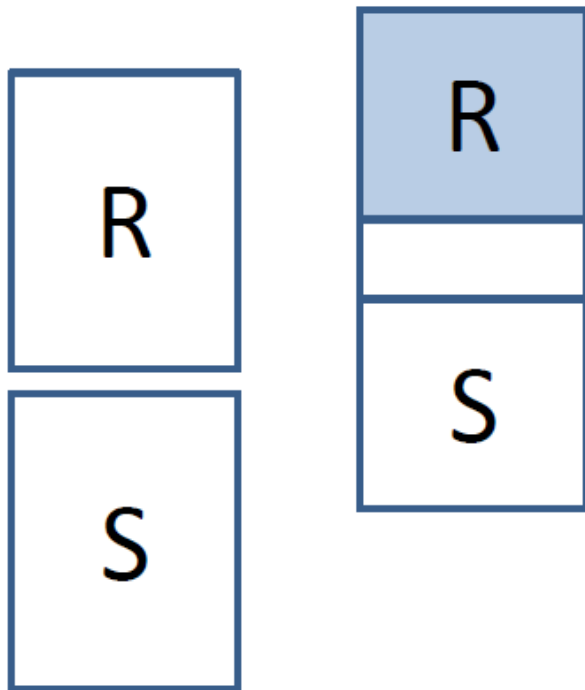


R (R1 INTERSECT R2)

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти

Разность (Set difference)

- Разностью отношений R и S (в указанном порядке) является отношение того же типа с телом, которое состоит из всех кортежей, присутствующих в R , но не присутствующих в S .
- Отношения R и S должны принадлежать к **одному типу**. Они должны быть совместимыми по объединению (union compatible), т. е. иметь одинаковые заголовки.
- Важно: оператор MINUS характеризуется направленностью (некоммутативностью), так же, как вычитание в обычной арифметике.



- К. Дейт использует такое обозначение:
 $R \text{ MINUS } S$
- Некоторые другие авторы — такое:
 $R - S$

Пример.

Students1 MINUS Students2

(предположим, что оба эти отношения имеют тот же тип, что и отношение Students)

Разность (Set difference)

Разностью отношений R_1 и R_2 называется отношение, содержащее множество кортежей, принадлежащий R_1 и не принадлежащий R_2 .

R_5 содержит перечень деталей, изготавливаемых только на участке 1, отношение R_6 – только на участке 2.

R_5

Шифр	Название детали
11075	Гайка М2
11033	Болт М1
13063	Шайба М1
13066	Шайба М3

R_6

Шифр	Название детали
11077	Гайка М4
11004	Болт М2

$$R_5 = R_1 \setminus R_2$$

$$R_5 = R_2 \setminus R_1$$

Операции объединения и пересечения являются **коммутативными**, т.е. результат не зависит от порядка следования аргументов в операции.

Операция разности является **несимметричной**, т.е. результат операции будет различным для различного порядка аргументов.

В отличие от навигационных средств манипулирования данными в теоретико-графовых моделях операции реляционной алгебры позволяют качественно иной результат:

1. Операции сразу целиком выполняются на отношениях. Для пользователя и прикладного программиста это означает, что в результате таких операций он сразу получит результат. Следовательно, пользователю не придется «вручную» последовательно перемещаться по базе данных в поисках необходимых записей, а программисту не придется писать отдельную программу для простейшей обработки данных.

2. Результатом выполнения операции является новое отношение, отражающее состояние исходного отношения в определенной части.

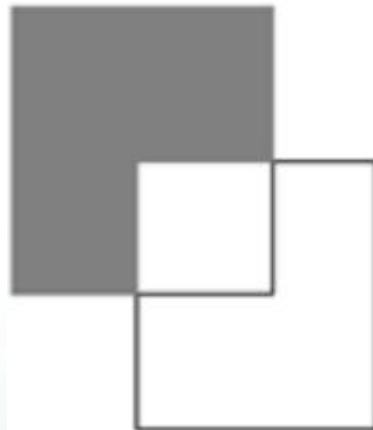
Разность (Set difference)

R1

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C3	Титов	Тольятти
C4	Павлов	Москва

R2

Код студента	Фамилия	Город
C1	Васильев	Самара
C2	Перов	Тольятти
C5	Сидоров	Тольятти
C6	Афанасьев	Москва



R1 - R2

R (R1 *MINUS* R2)

Код студента	Фамилия	Город
C3	Титов	Тольятти
C4	Павлов	Москва

Декартово произведение (Cartesian product) (1)

- Декартово произведение отношений R и S , не имеющих общих атрибутов – это отношение, заголовок которого представляет собой (теоретико-множественное) объединение заголовков отношений R и S , а тело состоит из всех кортежей t , таких, что t является (теоретико-множественным) объединением кортежа, принадлежащего к отношению R , и кортежа, принадлежащего к отношению S . Следует отметить, что кардинальность результата равна произведению кардинальностей входных отношений, R и S , а степень результата — сумме степеней входных отношений.
- Если необходимо сформировать декартово произведение двух отношений, имеющих общие имена атрибутов, то следует вначале воспользоваться оператором RENAME, чтобы переименовать атрибуты должным образом.
- К. Дейт использует такое обозначение: $R \text{ TIMES } S$
- Некоторые другие авторы — такое: $R \times S$

Декартово произведение (Cartesian product) (2)•

Отношение R

Last_name	First_name
Иванов	Петр
Петров	Иван

Отношение S

Discipline	Exam_date
Web-программирование	12.01.2018
Базы данных	15.01.2018
Язык C	21.01.2018

Отношение R × S

Last_name	First_name	Discipline	Exam_date
Иванов	Петр	Web-программирование	12.01.2018
Иванов	Петр	Базы данных	15.01.2018
Иванов	Петр	Язык C	21.01.2018
Петров	Иван	Web-программирование	12.01.2018
Петров	Иван	Базы данных	15.01.2018
Петров	Иван	Язык C	21.01.2018

Декартово произведение (Cartesian product) (3)•

R1

Код студента	Фамилия
С3	Титов
С4	Павлов

R2

Код дисциплины	Название
Д1	Информатика
Д2	Базы данных
Д3	Физика

R (R1×R2)

Код студента	Фамилия	Код дисциплины	Название
С3	Титов	Д1	Информатика
С3	Титов	Д2	Базы данных
С3	Титов	Д3	Физика
С4	Павлов	Д1	Информатика
С4	Павлов	Д2	Базы данных
С4	Павлов	Д3	Физика

Декартово произведение (Cartesian product) (4)•

Отношение R

A	B	C
a	b	c
c	a	d
c	b	d

Отношение S

D	E
1	3
2	4

Декартово произведение R×S

A	B	C	D	E
a	b	c	1	3
a	b	c	2	4
c	a	d	1	3
c	a	d	2	4
c	b	d	1	3
c	b	d	2	4

Соединение (Join) (1)•

- Соединение равносильно выполнению операции выборки (selection) над результатом декартова произведения: $R \text{ JOIN } S = (R \text{ TIMES } S) \text{ WHERE } p$
- Имеются следующие виды соединений:
 - θ -соединение
 - Эквисоединение (частный случай θ -соединения)
 - Естественное соединение
 - Внешнее соединение
 - Полусоединение
- **θ -соединение** определяет отношение, которое содержит кортежи из декартова произведения отношений R и S , удовлетворяющие предикату p . Предикат p имеет форму $R.a_i \theta S.b_j$, где θ может являться одним из операторов сравнения ($<, \leq, >, \geq, =, \neq$), а $R.a_i$ и $S.b_j$ – атрибуты отношений R и S соответственно.
- **Эквисоединение** – это частный случай θ -соединения. Он имеет место, когда в качестве оператора сравнения используется только проверка на равенство ($=$).
- **Естественное соединение** – это частный случай эквисоединения. Он имеет место, когда отношения R и S соединяются по всем общим атрибутам. Один экземпляр каждого из общих атрибутов исключается из результирующего отношения.

Соединение (Join) (2)•

Естественное соединение

Таблицы «Студенты» (students) и «Успеваемость» (progress)

Общий атрибут – record_book

Students JOIN Progress эквивалентно следующему выражению:

```
( Students TIMES ( Progress RENAME record_book AS p_record_book )  
  WHERE record_book = p_record_book )
```

Степень результата естественного соединения равна сумме степеней отношений R и S минус число общих атрибутов этих отношений.

Номер зачетной книжки	Ф. И. О.	Серия паспорта	Номер паспорта	Предмет	Учебный год	Семестр	Оценка
55500	Иванов Иван Петрович	0402	645327	Физика	2017/2018	1	5
55500	Иванов Иван Петрович	0402	645327	Математика	2017/2018	1	4
55800	Климов Андрей Иванович	0402	673211	Физика	2017/2018	1	4
55800	Климов Андрей Иванович	0402	673211	Физика	2017/2018	2	5

Соединение (Join) (3)

Внешнее соединение

Таблицы «Студенты» (students) и «Успеваемость» (progress)

Общий атрибут – record_book

Левое внешнее соединение отношений R и S – это соединение, в котором кортежи из R, которые не имеют соответствующих значений в общих атрибутах отношения S, также включаются в результирующее отношение. Отсутствующие значения атрибутов отношения S заменяются на NULL.

Номер зачетной книжки	Ф. И. О.	Серия паспорта	Номер паспорта	Предмет	Учебный год	Семестр	Оценка
55500	Иванов Иван Петрович	0402	645327	Физика	2017/2018	1	5
55500	Иванов Иван Петрович	0402	645327	Математика	2017/2018	1	4
55800	Климов Андрей Иванович	0402	673211	Физика	2017/2018	1	4
55800	Климов Андрей Иванович	0402	673211	Физика	2017/2018	2	5
55865	Новиков Николай Юрьевич	0202	554390				

Правое внешнее соединение определяется аналогично. **Полное внешнее соединение** сохраняет в результирующем отношении кортежи из обоих исходных отношений, заменяя отсутствующие значения на NULL.

Соединение (Join) (4)

- **Полусоединение** определяет отношение, которое содержит кортежи из отношения R, которые участвуют в соединении R и S, удовлетворяющие предикату p.
- R SEMIJOIN S
- Полусоединение выполняет соединение двух отношений, а затем выполняет проекцию по атрибутам первого операнда.
- Таблицы «Студенты» (students) и «Успеваемость» (progress)
Общий атрибут – record_book
(Students TIMES (Progress RENAME record_book AS p_record_book)
WHERE record_book = p_record_book AND discipline = 'Математика')
{record_book, name, psp_ser, psp_name}

Номер зачетной книжки	Ф. И. О.	Серия паспорта	Номер паспорта
55500	Иванов Иван Петрович	0402	645327

DEPT	DEPT #	DNAME	BUDGET
	D1	Marketing	10M
	D2	Development	12M
	D3	Research	5M

EMP	EMP #	ENAME	DEPT #	SALARY
	E1	Lopez	D1	40K
	E2	Cheng	D1	42K
	E3	Finzi	D2	30K
	E4	Saito	D2	35K

Рис. 3.1. База данных отделов и служащих (приведенные в ней данные часто используются в качестве примера)

Операция **сокращения** извлекает указанные строки из таблицы (в названии этой операции подразумевается, что кардинальность ее результата меньше или равна кардинальности исходной таблицы). Следует отметить, что операцию сокращения иногда называют выборкой; мы предпочитаем термин сокращение, поскольку соответствующий оператор не полностью соответствует оператору SELECT языка SQL.

Операция **проекции** предназначена для извлечения определенных столбцов из таблицы

Операция **соединения** предназначена для получения комбинации двух таблиц на основе общих значений в общих столбцах.

Сокращение:

DEPTs where BUDGET > 8M

Результат:

DEPT #	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M

Проекция:

DEPTs over DEPT#, BUDGET

Результат:

DEPT #	BUDGET
D1	10M
D2	12M
D3	5M

Соединение:

DEPTs and EMPs over DEPT#

Результат:

DEPT#	DNAME	BUDGET	EMP#	ENAME	SALARY
D1	Marketing	10M	E1	Lopez	40K
D1	Marketing	10M	E2	Cheng	42K
D2	Development	12M	E3	Finzi	30K
D2	Development	12M	E4	Saito	35K

Операция соединения. Прежде всего, заслуживает внимания то, что в таблицах DEPT и EMP есть общий столбец **DEPT#**, а следовательно, для этих таблиц можно выполнить операцию **соединения на основе общих значений в этом столбце**. При выполнении данной операции строка таблицы DEPT соединяется со строкой таблицы EMP и образуется более длинная строка, но подобное происходит тогда и только тогда, когда эти две строки имеют одинаковое значение поля **DEPT#**.

Сокращение:	Результат:	<table><tr><th>DEPT #</th><th>DNAME</th><th>BUDGET</th></tr><tr><td>D1</td><td>Marketing</td><td>10M</td></tr><tr><td>D2</td><td>Development</td><td>12M</td></tr></table>	DEPT #	DNAME	BUDGET	D1	Marketing	10M	D2	Development	12M
DEPT #	DNAME	BUDGET									
D1	Marketing	10M									
D2	Development	12M									
DEPTs where BUDGET > 8M											

Проекция:	Результат:	<table><tr><th>DEPT #</th><th>BUDGET</th></tr><tr><td>D1</td><td>10M</td></tr><tr><td>D2</td><td>12M</td></tr><tr><td>D3</td><td>5M</td></tr></table>	DEPT #	BUDGET	D1	10M	D2	12M	D3	5M
DEPT #	BUDGET									
D1	10M									
D2	12M									
D3	5M									
DEPTs over DEPT#, BUDGET										

Соединение:																														
DEPTs and EMPs over DEPT#																														
Результат:																														
<table><tr><th>DEPT#</th><th>DNAME</th><th>BUDGET</th><th>EMP#</th><th>ENAME</th><th>SALARY</th></tr><tr><td>D1</td><td>Marketing</td><td>10M</td><td>E1</td><td>Lopez</td><td>40K</td></tr><tr><td>D1</td><td>Marketing</td><td>10M</td><td>E2</td><td>Cheng</td><td>42K</td></tr><tr><td>D2</td><td>Development</td><td>12M</td><td>E3</td><td>Finzi</td><td>30K</td></tr><tr><td>D2</td><td>Development</td><td>12M</td><td>E4</td><td>Saito</td><td>35K</td></tr></table>	DEPT#	DNAME	BUDGET	EMP#	ENAME	SALARY	D1	Marketing	10M	E1	Lopez	40K	D1	Marketing	10M	E2	Cheng	42K	D2	Development	12M	E3	Finzi	30K	D2	Development	12M	E4	Saito	35K
DEPT#	DNAME	BUDGET	EMP#	ENAME	SALARY																									
D1	Marketing	10M	E1	Lopez	40K																									
D1	Marketing	10M	E2	Cheng	42K																									
D2	Development	12M	E3	Finzi	30K																									
D2	Development	12M	E4	Saito	35K																									

Таблица 3.1. Строки таблиц EMP и DEPT перед соединением

DEPT#	DNAME	BUDGET	EMP#	ENAME	DEPT#	SALARY
D1	Marketing	10M	E1	Lopez	D1	40K

Таблица 3.2. Результат соединения

DEPT#	DNAME	BUDGET	EMP#	ENAME	SALARY
D1	Marketing	10M	E1	Lopez	40K

Например, можно соединить в результирующую строку (табл. 3.1) следующие строки таблиц DEPT и EMP (названия столбцов приведены для наглядности). Это возможно, так как в общем столбце рассматриваемых строк имеется одно и то же значение D1. Результирующая строка приведена в табл. 3.2. Общий результат состоит из множества всех таких соединенных строк. Обратите внимание, что столбец DEPT# в каждой результирующей строке встречается один раз, а не два. Следует также отметить, что в поле DEPT# таблицы EMP отсутствует значение D3 (т.е. нет служащих, работающих в отделе D3), поэтому в данном поле нет и результирующих строк со значением D3, хотя строка со значением D3 в таблице DEPT присутствует.

Примеры

exams (name := 'Анна', course := 'Базы данных', grade := 5)

exams (name := 'Анна', course := 'Анализ данных', grade := 5)

exams (course := 'Анализ данных', grade := 4, name := 'Виктор')

exams (name := 'Нина', grade := 5, course := 'Базы данных')

Предположим, что отношение exams имеет атрибуты {name, course, grade}, принимающие значения в соответствующих доменах. Тогда истинное значение предиката на следующих кортежах указывает на то, что упомянутые студенты получили указанные оценки по этим курсам.

exams		
name	course	grade
Анна	Базы данных	5
Анна	Анализ данных	5
Виктор	Анализ данных	4
Нина	Базы данных	5

Рис. 2.2.1. Пример отношения в табличной записи

Успех реляционной модели данных, интенсивно используемой на протяжении десятилетий, обусловлен тем, что в рамках этой модели определены мощные и выразительные языки манипулирования данными. Один из таких языков строится на основе операций, позволяющих определять новые отношения из уже имеющихся. Это делает возможным использование результатов выполнения операций в качестве аргументов для выполнения следующих операций, т. е. задание требуемых вычислений в форме выражений.

Примеры

Операция проекции **PROJ** включает в результат подмножество атрибутов отношения, переданного в качестве аргумента. На рис. 2.2.2 показан результат проекции отношения exams на подмножество атрибутов {name, course}. Такой результат показывает только факт сдачи экзамена, но не оценку.

exams		
name	course	grade
Анна	Базы данных	5
Анна	Анализ данных	5
Виктор	Анализ данных	4
Нина	Базы данных	5

Рис. 2.2.1. Пример отношения в табличной записи

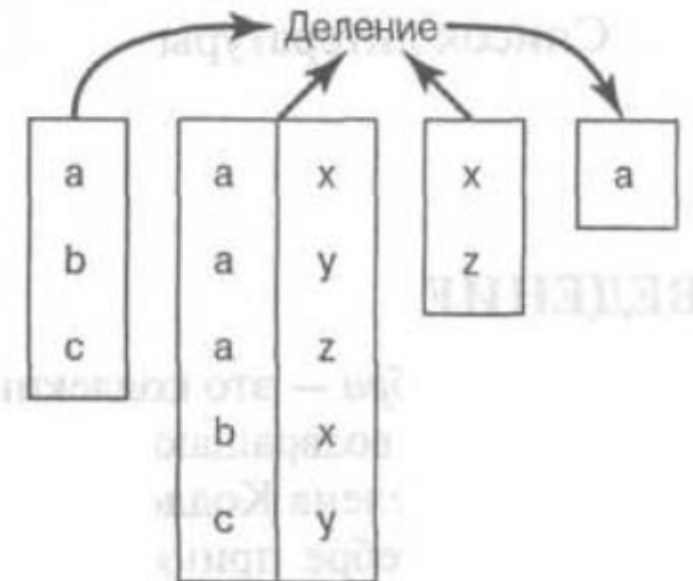
Успех реляционной модели данных, интенсивно используемой на протяжении десятилетий, обусловлен тем, что в рамках этой модели определены мощные и выразительные языки манипулирования данными. Один из таких языков строится на основе операций, позволяющих определять новые отношения из уже имеющихся. Это делает возможным использование результатов выполнения операций в качестве аргументов для выполнения следующих операций, т. е. задание требуемых вычислений в форме выражений.

Деление (Division)

- Предположим, что отношения R и S , соответственно, имеют следующие атрибуты: $X_1, X_2, \dots, X_m$ и $Y_1, Y_2, \dots, Y_n$. Здесь ни один из атрибутов X_i ($i = 1, 2, \dots, m$) не имеет одинакового имени с любым из атрибутов Y_j ($j = 1, 2, \dots, n$). Пусть отношение T имеет следующие атрибуты: $X_1, X_2, \dots, X_m$ и $Y_1, Y_2, \dots, Y_n$. Это означает, что T имеет заголовок, представляющий собой (теоретико-множественное) объединение заголовков отношений R и S . Будем рассматривать множества $\{X_1, X_2, \dots, X_m\}$ и $\{Y_1, Y_2, \dots, Y_n\}$, соответственно, как составные атрибуты X и Y . В таком случае операция деления R на S по T (где R — делимое, S — делитель, а T — посредник) может быть представлена с помощью следующего выражения:

$R \text{ DIVIDE BY } S \text{ PER } T$

Результат представляет собой отношение с заголовком $\{X\}$ и телом, состоящим из таких кортежей $\{X\}$, присутствующих в R , что кортеж $\{X\ x, Y\ y\}$ присутствует в T для всех кортежей $\{Y\ y\}$, присутствующих в S . Иными словами, данный результат состоит из тех значений X , присутствующих в R , для которых соответствующие значения Y в T включают все значения Y из S .



Специальные реляционные операторы

Выборка ($R \text{ WHERE } f$) отношения R по формуле f представляет собой новое отношение, заголовок которого совпадает с заголовком отношения R , а тело содержит множество кортежей, являющихся подмножеством множества кортежей отношения R , для которых формула f принимает значение истина.

Здесь f – логическая формула (предикат), принимающая значение «истина» или «ложь», образованная операндами, представляющими собой имена атрибутов отношений R или литералы, а также скалярными операторами сравнения ($=$, $<>$, $>$, $<$, $>=$, $<=$) и логическими операторами И, ИЛИ, НЕ. Выбор – унарная операция.

Пример. **Выборка**

На рис. представлено исходное отношение R и два отношения, полученные с помощью операторов:

1) $R \text{ WHERE Курс} = 1$;

2) $R \text{ WHERE Факультет} = \text{«Информатики»}$
 $\text{AND Курс} = 3$.

R

Код студента	Фамилия	Факультет	Курс
001017	Васильев	Информатики	1
002016	Перов	Математики	2
009017	Титов	Электроники	1
087015	Павлов	Информатики	3
005017	Афанасьев	Математики	1

$R \text{ WHERE Курс} = 1$

Код студента	Фамилия	Факультет	Курс
001017	Васильев	Информатики	1
009017	Титов	Электроники	1
005017	Афанасьев	Математики	1

$R \text{ WHERE Факультет} = \text{«Информатики»} \text{ AND Курс} = 3$

Код студента	Фамилия	Факультет	Курс
087015	Павлов	Информатики	3

Реляционное исчисление (1)

- Реляционное исчисление получило название от той части символической логики, которая называется *исчислением предикатов*. В контексте баз данных оно существует в двух формах: в форме предложенного Э. Коддом *реляционного исчисления кортежей* и в форме предложенного Лакруа и Пиро *реляционного исчисления доменов*.
- В логике первого порядка (или теории исчисления предикатов) под *предикатом* подразумевается истинностная функция с параметрами. После подстановки значений вместо параметров функция становится выражением, называемым *суждением*, которое может быть истинным или ложным.
- Например, предложения «Иван Петров является сотрудником данной организации» и «Иван Петров имеет более высокую зарплату, чем Петр Иванов» являются суждениями, поскольку можно определить их истинность или ложность. В первом случае функция «является сотрудником данной организации» имеет один параметр («Иван Петров»), а во втором случае функция «имеет более высокую зарплату, чем» имеет два параметра («Иван Петров» и «Петр Иванов»).

Реляционное исчисление (2)

- Если предикат содержит переменную, например в виде « x является сотрудником этой организации», то у этой переменной должна быть соответствующая *область определения*. При подстановке вместо переменной x одних значений из ее области определения данное суждение может оказаться истинным, а при подстановке других — ложным.
- Если P — предикат, то множество всех значений переменной x , при которых P становится истинным, можно символически записать следующим образом:
$$\{x \mid P(x)\}$$
- Предикаты могут соединяться с помощью логических операций $\wedge$ (AND), $\vee$ (OR) и $\sim$ (NOT) с образованием составных предикатов.

Реляционное исчисление (3)

Задача: Выбрать номера поставщиков и названия городов, в которых находятся поставщики детали с номером P2

- Реляционная алгебра:

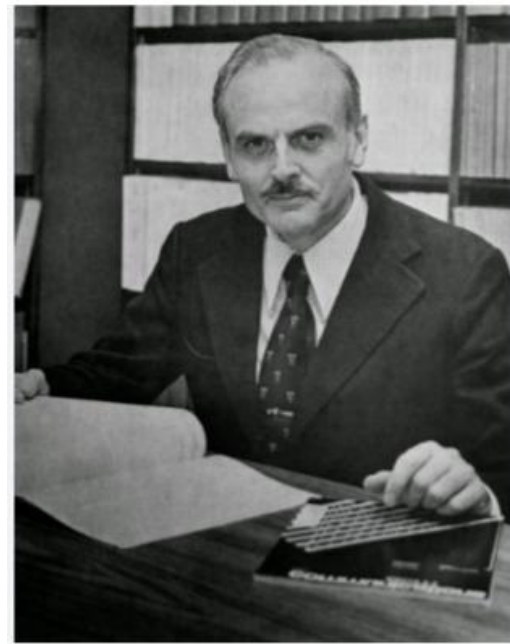
- Выполнить соединение отношений поставщиков S и поставок SP по атрибуту S#.
- С помощью операции сокращения выделить из результатов этого соединения кортежи, которые относятся к детали с номером P2.
- Сформировать проекцию результатов этой операции сокращения по атрибутам s# и CITY.

- Реляционное исчисление:

- Получить атрибуты s# и CITY для таких поставщиков, для которых в отношении SP существует запись о поставке с тем же значением атрибута s# и со значением атрибута P#, равным P2.
- В реляционном исчислении просто указывается, в чем заключается проблема, тогда как в реляционной алгебре задается *процедура решения* этой проблемы. На самом деле *реляционная алгебра и реляционное исчисление логически эквивалентны*.

12 правил Э.Ф.Кодда

В 1985 году в двух статьях в журнале Computer World Э.Ф. Кодд сформулировал правила, которым должны соответствовать настоящие реляционные базы данных. Всего правил было двенадцать. Несколько позже Кодд сформулировал 13-е правило и присвоил ему номер 0. Начнем рассмотрение правил с этого последнего или нулевого.



Правило 0 (фундаментальное правило). Реляционная система для управления базами данных должна использовать исключительно реляционные возможности. Другими словами для управления реляционными базами данных СУБД не может предоставлять какие-либо не реляционные операции. Данное правило является очень жестким и, к сожалению, нарушается многими СУБД. Даже всеми признанный и стандартизованный язык SQL or ['si:kwəl] содержит элементы «нереляционности». Можно сказать, что правило 0 требует безусловного исполнения следующих двенадцати правил.

12 правил Э.Ф.Кодда

Правило 1 (правило информации). Вся информация в реляционной базе данных (включая имена таблиц и столбцов) представляется в явном виде только на логическом уровне и только в виде значений, хранящихся в таблицах. Отсюда кстати вытекает и условие отсутствия порядка строк и столбцов в таблицах, поскольку упорядоченная последовательность добавляет дополнительную возможность хранения информации. Заметим также, что схема базы данных также должна храниться в табличном виде. Упоминание же в правиле 1 логического уровня означает, что информация об объектах более низкого уровня, например, индексах, должна быть исключена из модели данных.

Также следует заметить, что из первого правила, не явно вытекает и требование первой нормальной формы: атомарность данных, хранящихся на пересечении строк и столбцов таблицы. Действительно, если не требовать атомарности, тогда в столбцах могут храниться и сложные структуры, например те же таблицы. Поскольку данные одного столбца должны принадлежать одному и тому же домену, придется согласиться, что домен должен содержать множество таблиц, но эти таблицы также должны быть описаны соответствующими схемами и могут в свою очередь содержать таблицы. Вряд ли такую структуру можно назвать простой (явной) и удобной для проектирования. В результате мы можем получить иерархическую систему. Современные СУБД, однако, обходят требование первой нормальной формы и допускают хранения в столбцах таблиц. Мы вернемся к первой нормальной форме в разделе Нормальные формы.

12 правил Э.Ф.Кодда

Правило 2 (правило гарантированного доступа). Логический доступ ко всем и каждому элементу данных в реляционной базе данных обеспечивается комбинацией имени таблицы, имени столбца и значением первичного ключа. Это требование предполагает:

- о Уникальность имени таблицы в базе данных.
- о Уникальность имени столбца в таблице.
- о Уникальность первичного ключа, по крайней мере, в пределах одной таблицы.

В реальных СУБД могут присутствовать дополнительные параметры доступа, например имя пользователя, владельца данной базы. Кроме этого, поскольку система может работать с несколькими базами данных, в качестве еще одного параметра может быть имя базы данных. Наконец, если данные находятся под управлением разных СУБД (распределенная система) дополнительной координатой данных будет (адрес) СУБД.

12 правил Э.Ф.Кодда

Правило 3 (правило обработки неизвестных значений). В реляционной базе должна быть реализована возможность представлять неизвестные значения. Предполагается, что неизвестные значения отличаются от каких-либо определенных значений (числовых, строковых и т.п.) и должны быть реализованы для всех типов данных, хранящихся в базе. Данное правило провозглашает существование в реляционных базах данных значения **NULL**.

Правило 4 (правило доступа к словарю базы данных). Логическая структура словаря базы данных должна быть реляционной, чтобы пользователь, имеющий соответствующие права могли бы управлять структурой базы данных с помощью стандартного реляционного языка. Другими словами структура базы данных должна храниться в обычных реляционных таблицах.

12 правил Э.Ф.Кодда

Правило 5 (правило полноты языка управления данными). Должен существовать, по крайней мере, один язык управления реляционными базами данных, который поддерживал бы интерактивное и программное использование, и который должен быть представлен в виде набора команд, каждая из которых может быть представлена в виде одной строки. Такой язык должен поддерживать следующие возможности:

- о Определение структуры данных.
- о Определение представлений.
- о Модификацию данных (содержимого таблиц).
- о Определения условий целостности.
- о Определение правил авторизации (идентификация прав доступа).
- о Определение границ транзакций.

Это очень важное и жесткое правило, требующее наличия реляционного языка управления базами данных. Важным требованием к языку является то, что команды этого языка должны представляться в виде только одной строки. Данное требование вполне понятно, так как язык предполагалось использовать к удаленным данным (по каналам связи).

12 правил Э.Ф.Кодда

Правило 6 (правило обновления представлений). Все представления, которые теоретически можно обновить, должны быть обновляемы. Таким образом, для каждого создаваемого представления должен быть корректный алгоритм, позволяющий во время создания представления определить возможность вставки и удаления строк, а также столбцы, которые могут обновляться. Задача в действительности не столь проста, так как представление может состояться из данных от нескольких таблиц и, например, вставка строки должна предполагать вставку строки в несколько таблиц.

Представление в базе данных (БД) — это объект, который выглядит как таблица, но фактически не хранит данных, а извлекает их из других таблиц в момент обращения.

Представления иногда называют «виртуальными таблицами». Если данные в основной таблице меняются, пользователь получает актуальные данные при обращении к представлению, использующему эту таблицу.

Представления используются для следующих целей:

Упрощение сложных запросов. Они могут скрывать сложность структуры данных и предоставлять упрощённый интерфейс для доступа к данным.

Улучшение производительности. Создание представлений, которые инкапсулируют сложные запросы, может помочь оптимизировать выполнение этих запросов.

Обеспечение безопасности. Представления могут использоваться для обеспечения безопасности конфиденциальных данных. Создание представлений, которые ограничивают доступ к определённым столбцам или строкам данных, позволяет администраторам ограничить доступ к чувствительной информации.

12 правил Э.Ф.Кодда

Правило 7 (правило множественности обновлений). Операции вставки, удаления и обновления должны быть применимы к базовым таблицам, как и чтение данных из таблицы. Данное правило предполагает, что все операции модификации должны иметь и множественный вариант – операция может производиться не только над одной строкой таблицы, но и над группой строк.

Правило 8 (правило независимости на физическом уровне). Какие бы изменения на физическом уровне не происходили с данными или аппаратной частью, это не должно сказаться на функционировании прикладных программ или утилит управления данными. Из данного правила фактически и вытекает основное требование к СУБД, о котором мы уже говорили ранее. СУБД так должна взаимодействовать с операционной системой и, посредством нее с файловой системой, что изменения на файловом и аппаратном уровне не должны сказаться на функционировании ИС, которая построена на базе данной СУБД. Разумеется, это не означает, что для данной ИС не может быть специальных требований к ресурсам и аппаратуре. Но эти требования обусловлены особенностями функционирования данной ИС, а не зависимостью СУБД от среды исполнения.

12 правил Э.Ф.Кодда

Правило 9 (правило независимости на логическом уровне). Прикладное программное обеспечение не должно зависеть от изменений, вносимых в структуру базы данных, которые теоретически не должны изменять хранящиеся в базе данные. Данное требование связано с содержательной стороной структуры базы данных. Например, добавление в таблицу нового столбца не может сказаться на функционировании прикладных программ, так как доступ к данным осуществляется по именам столбцов. Порядок столбцов, который может при этом измениться не может влиять на доступ к данным (см. Правило 2).

Замечание

При добавлении столбца следует также учитывать команду вставки строки. Для того, чтобы не повлиять на эту команду для вставляемого столбца должно быть указано значение по умолчанию.

Напротив, удаление столбца из таблицы, скорее всего, приведет к неправильному функционированию системы, поскольку с большой вероятностью данный столбец явно (по имени) использовался в той или иной операции.

12 правил Э.Ф.Кодда

Правило 10 (правило независимости условий целостности). Должна существовать возможность формулировки правил целостности специфических для данной базы данных на языке реляционных баз данных. Правила целостности, таким образом, должны храниться в каталоге базы данных. Данное требование очень сильное и требует для полноты его исполнения наличие в частности в базе данных таких объектов как триггеры.

Правило 11 (правило независимости распространения). База данных может быть распределенной или переносится на другие компьютеры и это не должно сказаться на функционировании прикладного программного обеспечения. Иногда это правило формулируют как «независимость от потребностей пользователя», но такая формулировка, на наш взгляд, слишком неопределенна.

Правило 12 (правило единственности). Если в реляционной системе имеется низкоуровневый язык, то должна отсутствовать возможность использование его для того, чтобы обойти правила и условия целостности, сформулированные на реляционном языке и хранящиеся в каталоге базы данных.

Триггеры в базе данных — это специальные хранимые процедуры, которые автоматически выполняются при возникновении определённого события, такого как вставка, обновление или удаление данных из таблицы

Спасибо за внимание!